

**Université Toulouse III - Paul Sabatier
UT3**

**Rapport de stage
Master 1 Intelligence Artificielle : Fondements et Applications
Année universitaire 2023-2024**



Analyse des contraintes spécifiques aux interfaces tangibles

Auteur
CLOTILDE Anaïs

Tuteur professionnel
MENGIN Jérôme

Tuteur universitaire
GEORGE Jean-Pierre



Toulouse, du 2 mai au 31 juillet 2024

1 Remerciements

Je remercie Adrien DELMOTE, avec qui j'ai collaboré tout au long de nos stages. Nous nous sommes bien entraînés et nous avons pu apprendre l'un de l'autre.

Je remercie aussi notre tuteur professionnel, Jérôme MENGIN, ainsi que mes autres encadrants Florence BANNAY et Frédéric MARIS pour m'avoir donné l'opportunité de ce stage, d'avoir suivi l'évolution de mon travail et m'avoir apporté leurs retours. Je remercie aussi nos collaborateurs Christophe KOLSKI et Nadine COUTURE pour leurs conseils nous aidant à avancer dans le stage.

De plus, je remercie notre tuteur universitaire, Jean-Pierre GEORGE, pour ses informations sur le rapport et la soutenance de stage.

Enfin, je remercie mes proches pour m'avoir épaulée et conseillée durant ma recherche de stage ainsi que durant le stage lui-même.

Table des matières

1	Remerciements	1
2	Introduction	3
3	Présentation du stage	4
3.1	Présentation de l'établissement et de l'équipe	4
3.2	Contexte et objectifs du stage	4
3.3	Organisation du stage	4
4	Etat de l'art	5
4.1	Introduction	5
4.2	Interfaces tangibles	5
4.2.1	Définition d'une TUI	6
4.2.2	Différents types de TUI	7
4.2.3	Avantages et inconvénients des TUIs	11
4.3	Les CSPs	13
4.3.1	Base et applications du formalisme CSP	13
4.3.2	Extensions du formalisme CSP	13
4.3.3	Types de CSPs	14
4.4	Conclusion	17
5	Solveurs et CSPs	18
5.1	Ordonnancement des tâches	19
5.2	Répartition des employés de cinéma	19
5.3	n-reines	21
5.4	Problème d'horaire des infirmiers	24
6	Maquettes d'interface tangible	26
7	Conclusion	31
7.1	Bilan du travail effectué	31
7.2	Bilan personnel	31
7.3	Perspectives	31
8	Bibliographie	33
9	Annexes	36
9.1	Codes de résolution des problèmes de contraintes	36
9.1.1	Ordonnancement des tâches	36
9.1.2	Répartition des employés de cinéma	37
9.1.3	n-reines	39
9.1.4	Problème d'horaire des infirmiers	42
9.2	Logigramme représentant le fonctionnement de l'interface tangible	46

2 Introduction

Les problèmes de contraintes sont courants dans l'industrie. Parfois, le nombre de variables et de contraintes est tellement élevé que celles-ci sont difficiles à visualiser pour l'être humain. Pour résoudre cette situation, il est souvent possible d'écrire un programme permettant de résoudre ces problèmes, avec l'aide de solveurs. Cependant, tout le monde n'est pas informaticien. Et même si c'était le cas, la solution donnée par le solveur peut ne pas être satisfaisante. Si le solveur peut donner plusieurs solutions, alors le nombre de solutions trouvées peut être trop élevé pour que l'agent puisse aisément les consulter pour en choisir une satisfaisante.

Le travail effectué pendant le stage se place dans le cadre du projet TUI-Constraint (TUI = Tangible User Interface). Les personnes derrière ce projet pensent qu'une interface tangible pourrait pallier ce problème. En effet, la machine permettrait à l'agent de mieux visualiser les variables et contraintes d'un problème afin de trouver une solution satisfaisante. Aussi, l'interface tangible pourrait favoriser la collaboration entre différents agents cherchant ensemble une solution satisfaisante au problème.

Si plusieurs travaux ont déjà été réalisés sur les interfaces tangibles et les problèmes de contraintes, peu l'ont été pour mettre en évidence un lien entre ces deux problématiques.

J'ai fait mon stage à l'IRIT afin de contribuer au démarrage ce projet de recherche, j'étais en binôme avec Adrien DELMOTE également stagiaire de M1 sur ce même sujet. Notre travail consistait à effectuer un état de l'art, à développer des programmes pour résoudre des problèmes de contraintes, et à réaliser une maquette d'une interface tangible imaginée pour résoudre un problème précis.

3 Présentation du stage

3.1 Présentation de l'établissement et de l'équipe

Le stage s'est déroulé au sein de l'IRIT (Institut de Recherche en Informatique de Toulouse). Dirigé par Jean-Marc PIERSON, l'IRIT est un centre de recherches en informatique dont le site principal se trouve sur le campus de l'université Paul Sabatier. Ayant 600 membres permanents et non permanents, cet établissement a 7 départements de recherche : département ASR (Architecture, Systèmes, Réseaux) ; département CISO (Calcul Intensif, Simulation, Optimisation) ; département FSL (Fiabilité des Systèmes et des Logiciels) ; département GD (Gestion de Données) ; département IA (Intelligence Artificielle) ; département ICI (Intelligence Collective, Interaction) ; département SI (Signaux et Images). [16]

Notre tuteur professionnel Jérôme MENGIN et nos autres encadrants professionnels Florence BANNAY et Frédéric MARIS font partie de l'équipe ADRIA (Argumentation, Décision, Raisonnement, Incertitude et Apprentissage). Faisant partie du département IA, cette équipe de recherche se concentre sur les fondements théoriques de l'intelligence artificielle et plus particulièrement sur la représentation des connaissances et la formalisation du raisonnement. Nous collaborons aussi avec Nadine COUTURE (ESTIA de Bidart) et Christophe KOLSKI (université Valenciennes), qui ont déjà travaillé sur les interfaces tangibles dans le cadre de leurs recherches.

3.2 Contexte et objectifs du stage

Dans l'industrie, il est fréquent de rencontrer des problèmes de contraintes. Ils tendent à être difficiles à résoudre car ils impliquent un grand nombre de variables et de contraintes. Il est possible de demander à un solveur classique de résoudre ce problème, mais la solution que ce solveur peut donner peut ne pas être très satisfaisante pour les agents, surtout si ce n'est pas un problème d'optimisation.

Face à ces difficultés, le projet TUIConstraint vise à proposer, concevoir, réaliser et évaluer une interface tangible permettant la gestion d'espace de solutions de contraintes via la manipulation directe d'objets tangibles. Dans ce projet, on fait l'hypothèse que l'incarnation tangible des caractéristiques du problème et de l'espace de solutions facilitera les activités de décision. Aussi, on souhaite inciter les acteurs à coopérer et interagir avec l'interface tangible pour trouver des solutions par eux-mêmes. [21]

3.3 Organisation du stage

Durant le mois de mai, Adrien et moi avons fait des recherches sur les interfaces tangibles et les problèmes de contraintes pour pouvoir ensuite rédiger un état de l'art dessus. Par la suite, nous avons testé différents solveurs. D'autre part, un problème spécifique nous a été attribué. A partir de là, nous avons dû

nous imaginer une interface tangible permettant de résoudre le problème ainsi que son fonctionnement. Il a été envisagé qu'on écrive un programme en PDDL (Planning Domain Description Language) pour simuler les actions possibles de l'interface tangible. Cependant, nous avons manqué de temps pour nous en occuper. Nous avons conclu notre stage par une période de télétravail qui nous a permis de rédiger le rapport et de préparer le diaporama de soutenance de stage.

Nous avons fait des réunions hebdomadaires avec les encadrants pour leur permettre un suivi régulier de l'avancée de notre travail. De plus, nous avons effectué une réunion mensuelle avec Nadine COUTURE et Christophe KOLSKI pour leur faire part de notre travail du mois et obtenir leurs retours ainsi que leurs attentes sur la suite de notre travail.

Au début du mois de mai, nous avons créé un répertoire Github pour le stage https://github.com/DelmoteAdrien/stage_interfaces_tangibles. Nous y faisons régulièrement des commits, en mettant dans le répertoire nos notes de recherche, nos codes et nos documents de réunion. Cela nous permet de garder une trace de notre travail et de son avancée. Aussi, les encadrants y ont accès, ce qui les aide à suivre notre travail.

4 Etat de l'art

4.1 Introduction

Dans le cadre professionnel, notamment l'industrie, nous nous confrontons à différents problèmes impliquant un grand nombre de variables et de contraintes à respecter. Avec le programme adapté, une machine pourrait les résoudre. Cependant, l'humain a plus de difficultés à manipuler un grand nombre de variables et de contraintes. Il tend aussi à avoir des difficultés à insérer ce grand nombre de variables et de contraintes dans l'ordinateur. Enfin, il peut avoir des difficultés à trouver une solution satisfaisante en utilisant seulement l'ordinateur.

Face à ces difficultés, le projet TUIConstraint vise à proposer, concevoir, réaliser et évaluer une interface tangible permettant la gestion d'espace de solutions de contraintes via la manipulation directe d'objets tangibles. On fait l'hypothèse que l'incarnation tangible des caractéristiques du problème et de l'espace de solutions facilitera les activités de décision. Aussi, nous souhaitons inciter les acteurs à coopérer et à interagir avec l'interface tangible pour trouver des solutions par eux-mêmes.

Nous explorons donc les interfaces tangibles pouvant être utiles à ce projet ainsi que des manières de catégoriser des problèmes de contraintes et de les diviser en plusieurs sous-problèmes.

4.2 Interfaces tangibles

Le but de ce projet est de permettre aux acteurs de collaborer sur un problème de contraintes. Un bon moyen d'y arriver est de les réunir autour d'une

table pour discuter ou tester des solutions face au problème. Nous considérons les interfaces utilisateur tangibles (TUI) comme étant de bons outils pour atteindre cet objectif. Elles ont montré leurs bénéfices dans des domaines professionnels divers et variés, notamment pour les musées [17]. Néanmoins, il n’y a pas encore d’exploration sur leur utilité pour la résolution des problèmes de satisfaction de contraintes (CSP).

4.2.1 Définition d’une TUI

Les interfaces utilisateur graphiques (GUI) sont des interfaces transmettant graphiquement à l’utilisateur des données informatiques. Les plus courantes sont celles opérant dans l’écran d’un ordinateur. Figure 10, l’utilisateur peut manipuler les données informatiques, que ce soit avec des objets physiques (souris, clavier...) ou des éléments ne pouvant se manifester que sur l’écran (fenêtres, onglets...). Malgré l’utilisation possible d’objets physiques pour modifier ces données informatiques, ces données ne sont pas directement associées à ces objets physiques. Il y a donc une forte différence entre l’espace physique et l’espace virtuel.

Les interfaces utilisateur tangibles (TUI) sont des interfaces avec des objets tangibles - c’est-à-dire physiques et palpables - représentant des données informatiques dans le système. Les données physiques de ces objets sont converties en données informatiques dans le système. L’utilisateur peut alors modifier ces données en faisant bouger des objets. Parfois, le système peut aussi faire bouger des objets et ainsi modifier lui-même les données numériques associées. [6]

Selon le modèle contrôle-représentation (MCRpd, voir figure 1), les interfaces tangibles doivent respecter 4 propriétés fondamentales sur les représentations physiques (rep-p) [22] :

- Les représentations physiques sont reliées à des informations numériques dans le système. Les objets tangibles interagissant avec l’interface doivent être associés à des données numériques. L’utilisateur peut donc manipuler ces données à partir d’objets physiques. Par facilité, les objets peuvent représenter la valeur numérique ou l’utilité de la donnée numérique à laquelle ils sont couplés ;
- Les représentations physiques comportent des mécanismes de contrôle interactif. Même si certains objets tangibles peuvent rester physiquement immobiles et ne se déplacer que via la manipulation directe de l’utilisateur, d’autres peuvent être déplacés physiquement par le système ;
- Les représentations physiques sont en permanence associées à des représentations numériques médiatisées. En effet, les TUIs reposent sur un équilibre entre les représentations physiques et les numériques. Les représentations physiques jouent un rôle central dans la représentation et le contrôle des TUIs tandis que les représentations numériques tendent à assurer la médiation d’une grande partie des informations dynamiques fournies par le système informatique ;
- L’état physique des objets tangibles comporte des aspects clé de l’état numérique du système.

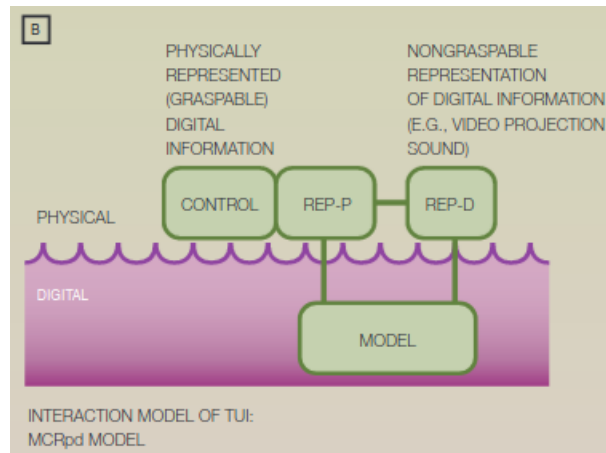


FIGURE 1 – Modèle MCRpd

4.2.2 Différents types de TUI

Les TUIs se divisent en 2 groupes : les interfaces statiques, qui laissent totalement l'utilisateur bouger les objets tangibles ; et les interfaces dynamiques, qui peuvent faire bouger eux-mêmes les objets tangibles. Il existe aussi une autre classification des TUIs en 4 types : les interfaces discrètes 2D, les interfaces 2.5D, les interfaces et outils cinétiques incarnés et les interfaces antigravité. (voir figure 2) [7]

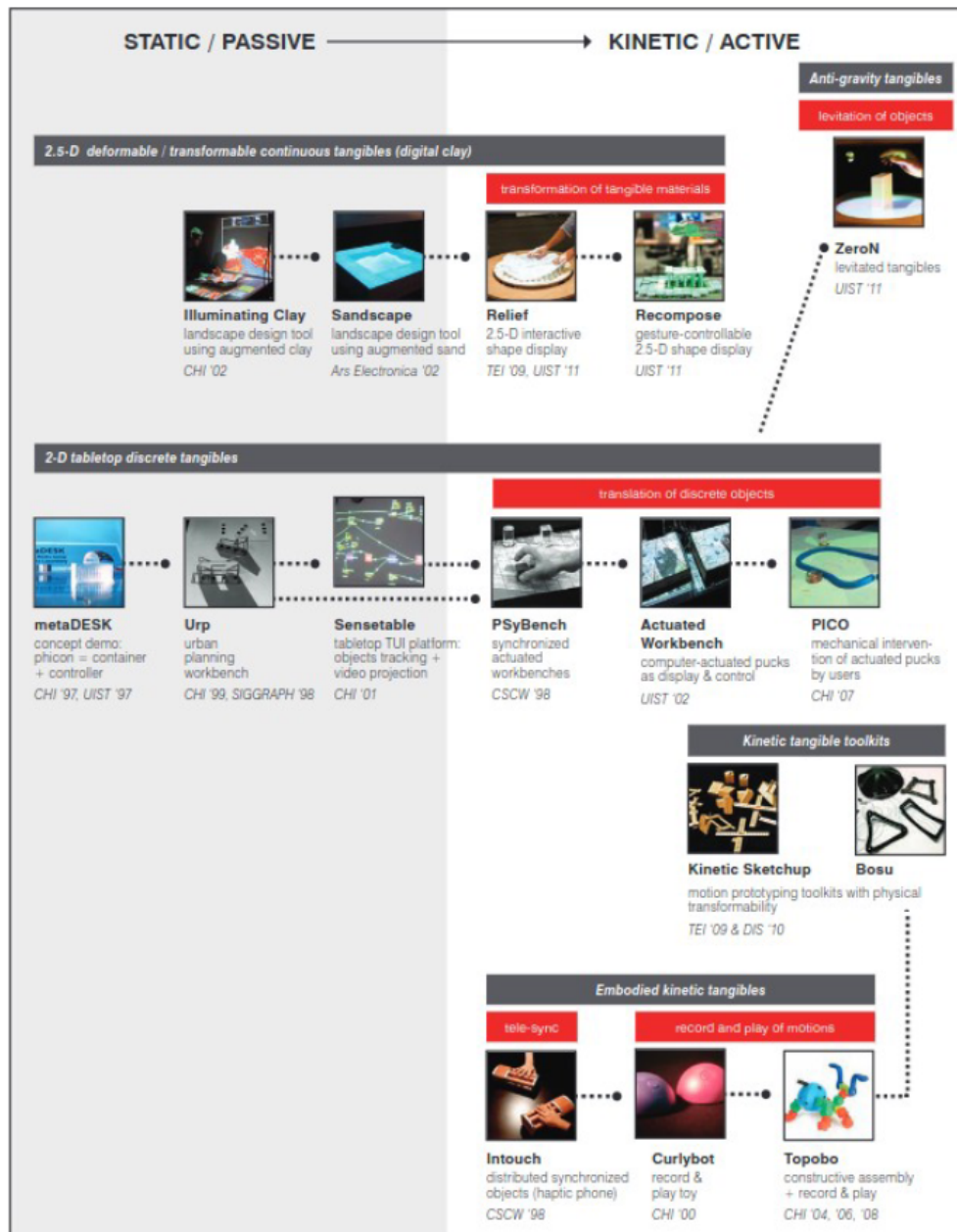


FIGURE 2 – Catégories des TUIs

Les interfaces discrètes en 2D sont des interfaces sur table qui ne sont pas du tout déformables. La manipulation des objets modifie les données numériques associées mais pas la forme de l'interface elle-même.

Un exemple de ce type de TUIs est MuseoTUI (voir figure 3). Il s'agit d'un plateau avec des figurines représentant les valeurs des caractéristiques des visiteurs d'un musée. L'utilisateur doit manipuler ces figurines afin de composer le profil d'un groupe de visiteurs pour lequel créer un parcours personnalisé de visite d'un musée. Deux figurines de même forme représentent une même caractéristique. Aussi, chaque colonne du plateau représente une caractéristique. Lorsque qu'un utilisateur place des figurines dans la zone de sélection (en haut), leurs caractéristiques sont ajoutées au profil et des LED indiquent leur niveau de traitement. Quand des figurines sont enlevées, leurs caractéristiques sont enlevées du profil. Dès qu'elles sont placées dans la zone de stockage (en bas), le système indique à quel point elles ont été prises en compte en combinaison avec le profil courant. [17]



FIGURE 3 – MuseoTUI

Un autre exemple de ce type de TUIs est la plateforme Sensetable (voir figure 4). Sur une surface digitale plane, Sensetable traque les positions de plusieurs objets avec un état digital donné. Cet état digital peut être modifié avec des tokens et des "boutons" de réglage. [15] Plusieurs applications et techniques d'interaction ont été développés à partir de Sensetable, comme Audiopad pour la composition de musique [14] et Supply Chain Visualization pour la simulation de modèles existants de chaînes d'approvisionnement. [3]

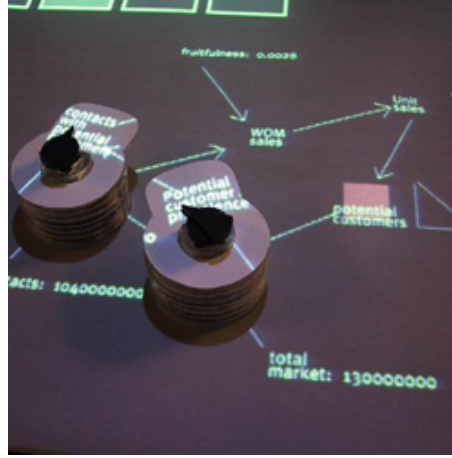


FIGURE 4 – Sensetable

Les interfaces continues déformables en 2.5D sont des TUIs sur table qui peuvent aussi se déformer. Les TUIs de ce type sont très rares et la plupart d'entre elles viennent du laboratoire MIT (Massachusetts Institute of Technology). Plusieurs de ces interfaces ont des tiges pouvant monter et descendre. Elles peuvent ainsi former des emplacements où placer des balles ou des boutons.

Un exemple de ce type de TUIs est inFORM (voir figure 5). Avec ses différentes tiges, inFORM peut former des ponts, des fentes, des rampes ou des surfaces. Ces formes peuvent être créées par l'utilisateur via la manipulation manuelle des tiges ou par le système via la manipulation automatique des tiges. Cette interface utilise aussi une caméra pour détecter des objets (généralement des balles) et un vidéoprojecteur pour afficher les informations du système. On peut utiliser inFORM pour plusieurs applications, notamment pour écouter des messages vocaux. [1]

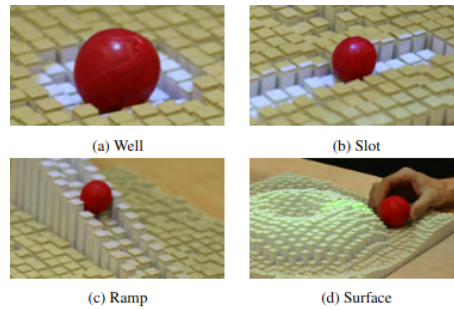


FIGURE 5 – inFORM

Les interfaces cinétiques incarnées et ensembles d'outils tangibles cinétiques reproduisent les mouvements qu'un utilisateur a réalisés sur un objet. La repro-

duction peut se faire sur le même objet ou un autre.

Parmi les outils tangibles cinétiques, on trouve notamment les Kinetic Cubes (voir figure 6), des cubes pouvant se déplacer, s’assembler et reproduire des mouvements sur une interface 2.5D avec des tiges avec l’aide de ces tiges et de divers objets associés. [18]

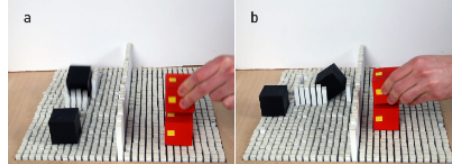


FIGURE 6 – Kinetic Blocks

Quant aux interfaces antigravité, elles permettent la lévitation et le déplacement libre des objets dans un espace 3D.

Nous nous sommes surtout intéressés aux 2 premiers types de TUIs. Plusieurs interfaces 2D et 2.5D pourraient être utiles pour notre projet, si nous pouvions les obtenir. Notamment, inFORM peut aider à la résolution de différents problèmes de contraintes impliquant le déplacement d’un objet, comme le cavalier d’Euler ; tandis que Sensetable pourrait permettre de résoudre des problèmes de coloration de graphes. D’ailleurs, les interfaces 2D tendent à favoriser la collaboration et la réflexion des agents.

Concernant les 2 derniers types, nous pensons que les Kinetic Blocks pourraient aider à la résolution des problèmes tel que celui de BlockWorld, sur une interface 2.5D avec des tiges comme inFORM. Cependant, nous n’avons pas trouvé d’autre TUI d’un de ces 2 types que nous pourrions utiliser pour notre projet.

4.2.3 Avantages et inconvénients des TUIs

Les avantages des interfaces tangibles pour la collaboration, notamment dans le contexte de l’apprentissage collectif, ont été beaucoup étudiés. Les TUIs favorisent la compréhension concrète des actions, la focalisation des utilisateurs, la conscience collective, l’externalisation, l’intuitivité et la participation active. [4] Aussi, trois propriétés des TUIs supportent la collaboration colocalisée [5] :

- 1- la familiarité des objets d’interaction abaisse le seuil d’engagement avec le système, ce qui augmente la probabilité que les utilisateurs collaborent ensemble ;
- 2- les points d’accès multiples permettent d’éviter tout goulot d’étranglement pour l’interaction, ce qui favorise l’usage parallèle et simultané du TUI ;
- 3- l’interaction avec des objets physiques est visible par toutes les personnes du groupe, ce qui permet une meilleure prise de conscience des données et une meilleure coordination.

Les TUIs favorisent aussi la réflexion tangible, qui est définie en 5 aspects [10] :

- 1- la "réflexion par l’action" décrit la façon dont la pensée et l’action sont profondément reliées pour coproduire apprentissage et raisonnement ;

- 2- la "performance" décrit la façon dont l'action physique peut être à la fois plus rapide et plus nuancée que la cognition symbolique ;
- 3- la "visibilité" décrit le rôle des artefacts dans la collaboration et la coopération, reprenant les arguments évoqués ci-dessus ;
- 4- le "risque" explore comment l'incertitude et la vulnérabilité corporelle façonnent la manière d'interagir entre être humains et avec les objets numériques, favorisant ainsi la confiance, l'engagement et la responsabilité ;
- 5- la notion de "densité de la pratique" suggère que la pratique réelle est souvent plus complexe que la modélisation informatique que les concepteurs peuvent en faire.

Aussi, les mouvements de la main et du bras favorisent la mémoire de travail et la mémoire spatiale. [8]

Il existe une distinction entre les actions pragmatiques, qui ont une conséquence fonctionnelle permettant d'atteindre un but ; et les actions épistémiques, qui n'ont aucune conséquence fonctionnelle mais permettent de mieux comprendre le problème. Prenons en exemple le jeu du Tetris. Le déplacement des pièces pour compléter une ligne est une action pragmatique, tandis que le pivotage des pièces pour explorer les différentes manières dont elles peuvent s'insérer est une action épistémique. Or, les actions épistémiques permettent d'explorer des options, de garder une trace de la réflexion et de diminuer la charge cognitive des utilisateurs en leur permettant d'utiliser des ressources externes à leur pensée. Via l'externalisation des données dans des objets physiques, les TUIs favorisent les actions épistémiques et donc la diminution de la charge cognitive. [9]

Cependant, malgré ces nombreux avantages des TUI, elles ont aussi des inconvénients. Un des inconvénients les plus évidents est que, comme tout objet physique, les objets tangibles prennent de la place et peuvent être perdus ou volés, surtout dans un environnement public. Aussi, les TUIs tendent à être rigides et statiques, donc peu malléables. [17] Notamment, annuler une action est bien plus compliqué à faire qu'avec le clavier ou la souris. Cependant, certains TUIs pallient ce problème, notamment les interfaces 2,5D. [1]

De plus, la comparaison des solutions est rendue plus complexe. Une difficulté courante avec les TUIs est celle d'étendre les solutions à des problèmes plus complexes traitant de grands nombres de paramètres ou de données. [19] Cela pose problème pour notre projet car les problèmes de contraintes en industrie tendent à impliquer un grand nombre de paramètres et de données. D'ailleurs, l'utilité des TUIs pour la résolution des CSPs n'a pas été beaucoup explorée. D'après nos recherches, seule Stéphanie Rey a fait une TUI pour résoudre un problème de contraintes. [17]

Enfin, les TUIs sont très coûteuses, en particulier les interfaces 2,5D. Elles peuvent être difficiles à créer, modifier et reproduire, comme les objets tangibles eux-mêmes. [17]

4.3 Les CSPs

Un CSP (Constraint Satisfaction Problem) est un problème mathématique à résoudre avec des contraintes à respecter. Des solveurs ont été développés pour en résoudre plusieurs. Il existe même des compétitions pour comparer les solveurs les plus performants pour résoudre un problème donné, notamment ceux des compétitions XCSP. [23] Dans la vraie vie, nous sommes aussi confrontés à ce type de problèmes. Dans le domaine professionnel, plusieurs de ces problèmes concernent un nombre élevé de variables et de contraintes. Souvent, ils ont aussi un grand nombre de solutions possibles. Or, cela complique leur résolution via une TUI. Nous souhaitons donc simplifier leur résolution via une TUI, notamment en les catégorisant et en les divisant en plusieurs sous-problèmes, afin que des agents puissent les résoudre avec une TUI dans le cadre professionnel.

4.3.1 Base et applications du formalisme CSP

Le CSP est un problème avec un ensemble de variables, de valeurs possibles pour ces variables et de contraintes sur ces variables. Le but est de résoudre le problème en respectant les domaines des variables et leurs contraintes.

Un CSP est défini par [2] :

- un ensemble fini de variables ;
- un ensemble fini de domaines discrets et finis des variables ;
- un ensemble fini de contraintes associées aux variables ;
- un ensemble de relations associées aux contraintes.

Une relation peut être exprimée de 2 manières [2] :

- en extension : utilisation d'un ensemble de valeurs permises pour des variables ;
- en intention : utilisation de prédicats ou de fonctions caractéristiques.

Des CSPs classiques comme celui des n-reines ou du coloriage des cartes sont étudiés dans le milieu académique. Les problèmes industriels comme la planification, l'ordonnancement et l'allocation des ressources sont aussi des CSP. En effet, ces problèmes comportent un ensemble fini de variables chacun associés à des domaines finis, ainsi qu'un ensemble fini de contraintes associées à ces variables. [2]

Il est possible d'aborder sous l'angle des CSPs divers problèmes d'intelligence artificielle et d'optimisation combinatoire, ainsi qu'un grand nombre d'applications industrielles. Parmi les problèmes d'intelligence artificielle et d'optimisation combinatoire, on trouve notamment l'ordonnancement et les contrôles de trafic aérien. Parmi les applications industrielles, on trouve notamment le génie civil, le génie mécanique, la cognition, les applications web, la sécurité des réseaux et la protection des données personnelles. [2]

4.3.2 Extensions du formalisme CSP

Les CSPs en industrie tendent à avoir beaucoup de variables et de contraintes. Cela augmente leur complexité, et par extension leur difficulté et temps de traitement. De plus, nous pouvons difficilement représenter ce grand nombre de variables et de contraintes. Par conséquent, nous devons trouver une façon de

réduire le nombre de variables et de contraintes que le TUI doit traiter, de sorte qu'on puisse résoudre le problème original par la suite. Nous devons aussi trouver une manière de pouvoir résoudre le même problème de contraintes partie par partie. Pour cette raison, nous avons exploré plusieurs extensions du formalisme CSP, parmi celles explorées dans la recherche. Ces extensions ont pour but d'élargir le champ d'action de ce formalisme en introduisant de nouveaux concepts. [2]

La plupart des problèmes, surtout réels, évoluent au fil du temps selon l'environnement et l'utilisateur. Face à ces changements, il faut trouver rapidement une nouvelle solution satisfaisante. Un CSP dynamique (DCSP) se compose d'une série $P_0, \dots, P_k$ de CSPs. Chaque CSP P_{i+1} vient d'une modification d'un problème P_i . Il peut s'agir d'un ajout ou d'une suppression d'une ou de plusieurs contraintes. La suppression d'une contrainte conserve les solutions mais pousse à réévaluer les déductions et éliminations de valeurs pour des variables de la version précédente du problème. L'ajout d'une contrainte conserve ces déductions et éliminateurs de valeurs mais pousse à éliminer des solutions de la version précédente du problème. [2]

Un CSP distribué est un CSP divisé en plusieurs sous-CSPs. Le but est de résoudre tous les sous-CSPs en satisfaisant leurs contraintes, puis de résoudre le CSP avec l'aide des solutions trouvées précédemment. [2]

Une des limites majeures du formalisme classique CSP est la rigidité du traitement des problèmes sur-contraints, menant souvent à l'absence de solution. Pour régler ce problème, une extension générique se base sur la relaxation du problème sur-contraint pour obtenir un problème équivalent plus simple et surtout solvable. La relaxation peut s'effectuer via la suppression d'une variable, le retrait d'une contrainte et l'extension du domaine d'une variable ou d'une contrainte. Un CSP partiel (PCSP) est composé [2] :

- du CSP de départ P ;
- de l'espace des problèmes $(PS, \leq)$ partiellement ordonnés contenant P tel que PS est un ensemble de CSP P_i et ' $\leq$ ' un ordre partiel sur ces problèmes. $P_i \leq P_{i+1}$ est vérifié si l'ensemble des solutions de P_{i+1} est inclus dans l'ensemble des solutions de P_i ;
- de la fonction de distance M sur $(PS, \leq)$;
- des bornes nécessaires et suffisantes (N, S) de la distance entre le problème d'origine P et un problème P' solvable dans PS .

Une solution d'un PCSP est donnée par un problème P' du PS et ses solutions, pour laquelle la distance entre P et P' est inférieure à N . Si la distance entre P et P' est minimale, alors on parle d'une solution optimale.

Un maximal CSP (Max-CSP) est une variante du PCSP dont le but est de satisfaire le plus de contraintes possibles. [2]

4.3.3 Types de CSPs

Nous avons cherché une classification de CSP qui pourrait nous aider à les représenter dans une TUI. Toutefois, même s'il existe des classifications de ces problèmes, nous avons des difficultés à nous en servir dans ce but-là. Par consé-

quent, nous vous proposons une nouvelle classification des CSP.

Les problèmes d'affectation sont des problèmes avec des contraintes temporelles ou spatiales. Le but est d'affecter des positions, spatiales ou temporelles, à des objets ou des agents. Techniquement, tous les CSPs sont des problèmes d'affectation. Dans cette classification, nous pouvons classer dans ce type les problèmes d'incompatibilités individuelles, c'est-à-dire ceux avec des variables ne pouvant pas avoir la même valeur que ceux avec qui ils sont reliés.

Un exemple de ce type de problèmes est celui des n -reines. Il consiste à placer n reines sur un échiquier de dimensions $n \times n$ de sorte à ce qu'elles ne puissent pas s'attaquer l'une l'autre. Selon les règles de l'échiquier, cela implique qu'elles ne soient pas dans la même ligne, ni dans la même colonne, ni dans les mêmes diagonales. [2]

On peut aussi trouver comme problème d'affectation celui des horaires d'infirmiers. Le but est de placer les infirmiers dans des services sur un certain nombre de jours, selon des critères temporels. Il existe plusieurs variations de ce type de problèmes, selon le nombre d'infirmiers, de jours et de services par jour, et aussi selon les contraintes posées. Parmi les contraintes les plus courantes, il y a celle que si un infirmier fait un service de nuit dans un jour donné, alors il ne doit pas faire de service du matin le lendemain. [13]

Les problèmes de planification ont les mêmes types de contraintes comme les problèmes d'affectation, mais le but est de planifier des actions pour arriver à un but précis. Donc pour ce type de problème, les affectations ne peuvent pas se faire dans n'importe quel ordre.

Un exemple de ce type de problèmes est celui du cavalier d'Euler. Il consiste à déplacer un cavalier sur un échiquier de façon à ce qu'il passe par toutes les cases une et une seule fois. [2] Le cavalier ne peut faire que des déplacements en L, c'est-à-dire : un déplacement de 2 cases verticalement et d'1 case horizontalement ou de 2 cases horizontalement et d'1 verticalement. Le parcours peut être ouvert ou fermé. Dans le dernier cas, le cavalier peut passer de la case de départ à la case d'arrivée en un unique déplacement. [11]

42	57	44	9	40	21	46	7
55	10	41	38	45	8	39	20
12	43	56	61	22	59	6	47
63	54	11	30	25	28	19	38
32	13	62	27	60	23	48	5
53	64	31	24	29	26	37	18
14	33	2	51	16	35	4	49
1	52	15	34	3	50	17	36

FIGURE 7 – Solution avec parcours fermé proposée par Euler pour un échiquier 8*8

Concernant **les problèmes de configuration**, nous avons des contraintes obligatoires et des préférences, et nous devons proposer une configuration selon ces contraintes et préférences. Dans la vraie vie, ce type de problèmes implique souvent de préparer pour un client un produit ou un service correspondant à un certain nombre de contraintes. Nous avons donc affaire à des incompatibilités de groupe en présence de préférences utilisateurs.

Un exemple de ce type de problèmes est celui de la préparation d'une visite personnelle d'un musée pour un groupe de personnes. Cette préparation doit se faire selon les critères obligatoires et les préférences d'un membre du groupe. Parmi les critères obligatoires, il y a le fait qu'un enfant de 3 ans ne peut pas faire la visite sans un adulte à ses côtés. C'est une problématique que Stéphanie Rey a abordée dans sa thèse. Elle propose un ensemble de 6 caractéristiques visiteurs : le nombre de personnes dans le groupe, l'âge, l'expérience, l'humeur, le handicap et la durée de la visite. À partir de cette classification des visiteurs, le MuseoTUI aide à la personnalisation d'une visite d'un musée pour un groupe de visiteurs. [17]

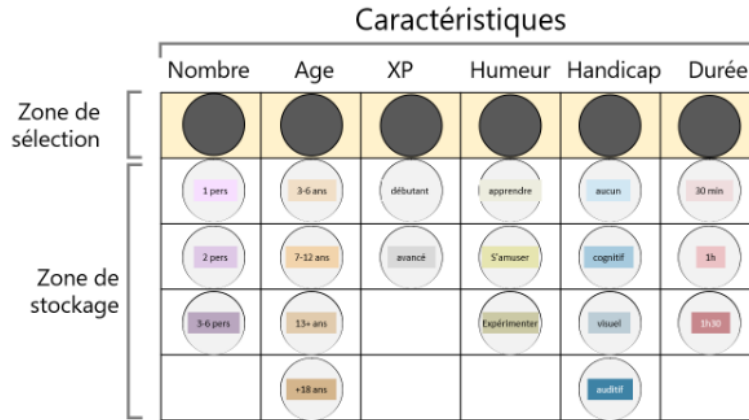


FIGURE 8 – Maquette de MuseoTUI avant toute configuration

Un autre exemple de ce type de problèmes est celui de la configuration de camions. Contrairement aux voitures, qui viennent souvent d'un modèle standard avec quelques variations, ces véhicules sont très dépendants des critères des clients, qui tendent à être spécifiques. Par exemple, une entreprise de produits laitiers peut demander un camion adapté aux montagnes et aux déserts ainsi qu'une cabine avec un grand espace. De plus, on ne trouve pas plusieurs véhicules identiques, sauf si le client l'a spécifiquement demandé. Le but ici est de satisfaire la demande des clients tout en restant conformes aux lois et aux normes d'ingénierie, de fabrication et de marketing. [12]

Notre classification comporte encore quelques limites. Il reste parfois une ambiguïté pour certains problèmes. Par exemple, le sudoku peut être considéré comme un problème d'attribution ou de configuration. De plus, la classification représente mal les manières possibles de représenter physiquement les contraintes des problèmes de même catégorie. Malgré ces défauts, cette classification nous a servi de départ pour nous faire une meilleure idée de la manière de représenter des problèmes d'une manière tangible. En effet, nous avons pu nous en servir pour réfléchir à la représentation tangible de ces problèmes de contraintes et de la manière de les résoudre.

4.4 Conclusion

Nous explorons l'hypothèse qu'une interface tangible pourrait aider des agents à collaborer ensemble pour résoudre un problème de contraintes à grande échelle dans l'industrie. En effet, le côté palpable des objets auxquels seront associés des variables et des contraintes permettrait de plus facilement visualiser ces variables et contraintes, ce qui aiderait à la résolution de ces problèmes en équipe. Après nos recherches sur les interfaces tangibles, nous avons décidé de nous concentrer sur les interfaces 2D. En effet, à part les interfaces 2.5D, nous voyons peu d'utilité des autres types d'interfaces tangibles pour la résolution des problèmes de contraintes. De plus, les interfaces 2.5D sont très coûteuses et leur

utilisation pour résoudre des problèmes de satisfaction de contraintes est très faible. Enfin, les interfaces 2D présentent une grande diversité qui pourrait nous servir pour le projet.

Cependant, un problème important dans notre projet est que les CSPs en industrie impliquent souvent un grand nombre de variables et de contraintes, qui seraient difficiles à représenter d'une manière tangible. Nous devons donc réfléchir à une manière de bien les représenter, qui correspondrait le mieux possible au problème et permettrait d'en trouver une solution satisfaisante pour l'équipe. D'une part, nous nous sommes penchés sur les extensions, qui pourraient aider à diviser un problème en ensemble de sous-problèmes ou à réduire durant un temps le nombre de variables et de contraintes à traiter, de sorte à ce qu'on puisse arriver à une solution satisfaisante pour tout le problème. D'autre part, nous avons réfléchi à une première classification des problèmes de contraintes qui nous aiderait à les représenter d'une manière tangible.

5 Solveurs et CSPs

Après avoir fait nos recherches pour l'état de l'art, nous avons parlé à nos partenaires Christophe KOLSKI et Nadine COUTURE de ce que nous avons trouvé et de ce que nous avons compris. Nous leur avons notamment parlé de cette première classification des problèmes de contraintes. En se basant là-dessus, il a été décidé qu'Adrien travaillerait sur les problèmes de configuration et moi sur les problèmes d'attribution.

J'ai donc écrit des codes pour résoudre plusieurs problèmes de contraintes, avec le langage Python et un des deux solveurs suivants : celui de la librairie `python-constraint`¹ puis un solveur d'OR-Tools (Google) appelé CP-SAT². Aussi, j'ai utilisé la librairie `time` pour calculer la durée de la recherche de solutions.

J'ai écrit des programmes pour résoudre les problèmes suivants :

- ordonnancement de tâches ;
- répartition des employés de cinéma ;
- n-reines ;
- problème de Shurr ;
- Sudoku ;
- coloration des graphes ;
- problème d'horaire des infirmiers.

Dans le répertoire Github, il est possible de les trouver dans le dossier suivant : https://github.com/DelmoteAdrien/stage_interfaces_tangibles/tree/main/code/Anais/Solveurs. Je vais vous décrire mes programmes pour résoudre les problèmes suivants : ordonnancement des tâches ; répartition des

1. Voici le lien pour en connaître des bases sur `python-constraint` et pour l'installer <https://pypi.org/project/python-constraint/> ainsi qu'un autre donnant de la documentation sur cette librairie <http://labix.org/doc/constraint/>

2. Voici le lien pour installer OR-Tools <https://developers.google.com/optimization/install?hl=fr> et celui pour connaître des bases sur CP-SAT https://developers.google.com/optimization/cp/cp_solver?hl=fr

employés de cinéma ; n-reines et problème d'horaire des infirmiers. Il est possible d'en voir les codes dans les annexes, avec un ajustement des commentaires pour qu'ils se voient mieux dans le rapport.

5.1 Ordonnancement des tâches

Pour ce problème comme pour le suivant, j'ai utilisé la librairie python-constraint. Le programme dont je vais vous parler peut se trouver ici, en plus des annexes.

Le principe de ce type de problèmes est le suivant : on a un ensemble de tâches avec des durées et des contraintes dessus. Le but est de les commencer ou de les finir avant un temps donné.

Pour mon programme, je me suis concentrée sur l'exercice n°3 de TD de CSP. Voici les consignes du début de l'exercice :

"On considère le problème d'ordonnancement suivant :

- on a 4 tâches à réaliser de durées 5, 1, 3, 4
- les tâches 2 et 3 ne peuvent être faites en parallèle (mêmes ressources nécessaires), et la 3 doit être faite avant la 4.

On veut savoir si toutes les tâches peuvent être commencées avant le temps $t=7$."

Partons du principe que les durées sont écrites en minutes. Nous pouvons définir les variables entières A, B, C et D représentant respectivement le début des tâches n°1, n°2, n°3 et n°4.

La première contrainte est que les tâches doivent commencer avant 7 minutes. Cela signifie que les variables représentant leurs débuts doivent être entre 0 et 6.

La deuxième contrainte est que les tâches n°2 et n°3 ne peuvent pas s'effectuer en même temps. Puisque la tâche n°2 dure 3 minutes et la tâche n°3 dure 3 minutes, cela signifie que nous avons $B \leq C - 1$ ou $C + 3 \leq B$.

La troisième contrainte est que la tâche n°3 doit être faite avant la tâche n°4. Vu que la tâche n°3 dure 3 minutes, cela signifie que nous avons $C + 3 \leq D$.

Je définis les variables et contraintes ci-dessus dans mon programme. Dès que ce dernier est exécuté, l'invite de commande donne 280 solutions en $1,59.10^{-3}$ secondes. J'ai ensuite cherché à réduire le nombre de solutions, et par extension à réduire le temps de recherche des solutions. Un moyen simple de le faire est de minimiser la fin des tâches.

Pour rappel, on a $C + 3 \leq D$. Si $C = 0$ alors on a $3 \leq D$ donc $7 \leq D + 4$. Si on pose $D = 3$, on peut obtenir $D + 4 = 7$ donc $D + 4 \leq 7$.

J'ajoute donc la contrainte que les tâches doivent se terminer en maximum 7 minutes. Ainsi, l'invite de commande montre 12 solutions en $7,59.10^{-4}$ s.

5.2 Répartition des employés de cinéma

Les problèmes de répartition des employés dans des salles consistent à répartir des personnes dans des salles selon des critères précis. Ce problème-là vient

de ce lien https://www.artelys.com/app/docs/kalis/3_solveCP.html parlant d'une autre librairie de solveurs. Le programme dont je vais vous parler peut se trouver ici, en plus des annexes.

On a affaire à 8 employés : Andrew, David, Jane, Jason, Leslie, Michael, Marilyn et Oliver. Il faut les répartir dans les salles suivantes : le guichet, la première entrée, la seconde entrée et les vestiaires. Ces salles requièrent respectivement 3, 2, 2 et 1 personne. Les contraintes sont les suivantes :

- chaque employé est placé dans un lieu unique (appelé "location" dans mon code) ;
- chaque lieu est occupé par le nombre d'employés requis ;
- Leslie travaille à la seconde entrée ;
- Michael travaille à la première entrée ;
- David, Michael et Jason ne peuvent pas travailler ensemble ;
- si David vend des tickets, alors Marilyn doit être avec lui.

J'ai dû réfléchir à la manière de représenter les places occupées par les employés. Voici ce que j'ai décidé pour la modélisation du problème.

Nous avons 8 variables : a, b, c, d, e, f, g, h. Voici ce qu'elles représentent :

- a, b et c : les places au guichet ;
- d et e : les places à la première entrée ;
- f et g : les places à la seconde entrée ;
- h : la place aux vestiaires.

Ces variables peuvent prendre un entier entre 0 et 7, représentant l'index du nom d'un employé dans le tableau suivant :

```
employes = ["Andrew", "David", "Jane", "Jason", "Leslie", "Michael", "Marilyn", "Oliver"]
```

Ces variables indiquent donc quels employés occupent les places représentées par ces variables. En plus d'indiquer où ces employés travaillent, je respecte ainsi le nombre de places demandé pour ces lieux.

Et voici comment je modélise les contraintes.

La première est que chaque employé se trouve dans un seul lieu à la fois. Cela signifie notamment que a et d ne peuvent pas avoir comme valeur 3, sinon cela signifierait que Jason travaille à la fois au guichet et à la première entrée.

Il y a aussi une autre chose à prendre en compte pour une bonne résolution du problème : chaque employé doit occuper une place. Cela signifie notamment que, même si b et c représentent 2 places d'un même lieu, ils ne peuvent pas prendre la même valeur, sinon au moins 1 employé sera manquant dans la solution.

Pour les raisons ci-dessus, j'ai décidé que toutes les variables seraient différentes.

Les deuxième et troisième contraintes sont que Leslie travaille à la seconde entrée et Michael à la première. L'indice des noms "Leslie" et "Michael" dans le tableau "employes" sont respectivement 4 et 5. De plus, d et e représentent les places à la première entrée alors que f et g représentent celles à la seconde. Par conséquent, d ou e doit être égal à 5 tandis que f ou g doit être égal à 4.

La troisième contrainte est que David, Michael et Jason ne peuvent pas travailler ensemble. D'une part, pour 2 variables représentant des places dans un même lieu, si l'une d'elles est égale à 1 (David), alors l'autre doit être différente

de 3 (Jason) et de 5 (Michael). D'autre part, pour 2 variables représentant des places dans un même lieu, si l'une d'elles est égale à 3 (Jason), alors l'autre doit être différente de 5 (Michael).

La dernière contrainte est que si David est au guichet, alors Marilyn doit y être aussi. Donc pour les variables a , b et c , si l'une d'elles est égale à 1 (David), alors l'une des 2 autres variables doit être égale à 6 (Marilyn).

Je définis ainsi les variables et contraintes du problème. J'obtiens alors 1620 solutions en $1,44.10^{-1}$ secondes. A l'affichage, je remarque un problème important de symétrie. En effet, plusieurs des solutions affichées sont des solutions identiques qui sont pourtant différentes selon l'ordinateur.

Voici 2 exemples de solutions identiques affichées séparément :

Solution n°1 :

Dans le guichet : Oliver, Marilyn et Jason

Dans la première entrée : Michael et Jane

Dans la seconde entrée : Leslie et Andrew

Dans les vestiaires : David

Et :

Solution n°3 :

Dans le guichet : Oliver, Marilyn et Jason

Dans la première entrée : Michael et Jane

Dans la seconde entrée : Andrew et Leslie

Dans les vestiaires : David

En effet, dans la solution n°1 on a $f = 4$ et $g = 0$, tandis que dans la solution n°3 c'est l'inverse. Or, variables f et g représentent les places de la seconde entrée. Pour cette raison, ces 2 solutions sont en réalité les mêmes. Pour d'autres cas de solutions identiques affichées comme étant distinctes, le problème est similaire.

Pour régler ce souci de symétrie, j'ai rajouté une contrainte faisant qu'on a $a < b < c$, $d < e$ et $f < g$. Avec cette contrainte supplémentaire, j'obtiens 36 solutions en $1,07.10^{-2}$ secondes.

5.3 n-reines

Pour ce problème et le suivant, j'ai utilisé la librairie permettant d'utiliser le solveur SAT d'OR-Tools, qui est un logiciel Open Source développé par Google. Aussi, la version cherchant plusieurs solutions se trouve ici tandis que celle n'en cherchant qu'une seule se trouve ici.

Soit n un entier naturel non nul. Sur un échiquier de dimensions $n \times n$, il faut placer n reines de sorte à ce qu'elles ne puissent pas s'attaquer l'une l'autre. Cela signifie qu'elles ne peuvent être :

- dans la même ligne ;
- dans la même colonne ;
- dans la même diagonale.

Voici comment je modélise le problème.

Soit n un entier naturel non nul représentant le nombre de reines. Nous avons alors n variables. Ici, pour tout i entier entre 1 et n , nommons les X_i . En réalité, je leur donne un autre nom dans le code.

Concernant les lignes, je pars du principe que pour tout i entier entre 0 et $n-1$, la reine $n^o i$ se trouve à la ligne $n^o i$. Ainsi, je n'ai pas besoin de définir la contrainte selon laquelle les reines ne doivent pas être dans la même ligne.

Pour la contrainte que les reines ne doivent pas être dans la même colonne, il suffit de la définir ainsi :

$$\forall i, j \in \llbracket 1 ; n \rrbracket i \neq j \Leftrightarrow X_i \neq X_j$$

Pour la contrainte que les reines ne doivent pas être dans la même diagonale, j'ai eu plus de difficultés à la définir. Après quelques temps, j'ai trouvé une manière de la définir dans le livre "Problèmes de satisfaction de contraintes : formalismes et techniques CSP" par K. Ghedira [2]. Voici les 2 propriétés à établir pour définir la contrainte :

$$\forall i, j \in \llbracket 1 ; n \rrbracket i \neq j \Leftrightarrow X_i + j \neq X_j + i$$

$$\forall i, j \in \llbracket 1 ; n \rrbracket i \neq j \Leftrightarrow X_i - j \neq X_j - i$$

Au début de l'exécution de ce programme, je permets à l'utilisateur de saisir un entier désignant le nombre de reines à mettre. Ici, il s'agit d'une action rapide à effectuer. On ne peut pas en dire autant des problèmes comme le sudoku ou la coloration de graphes. Pour le sudoku, il serait long de saisir dans l'invite de commande toutes les cases remplies dès le départ. Pour la coloration de graphes, selon le nombre de sommets et d'arêtes, leur saisie peut aussi être longue. Par conséquent, pour ces problèmes, il est préférable de privilégier la lecture d'un fichier par l'ordinateur.

Voici un petit aparté sur ma façon de nommer les variables par la suite. Comme dans le programme résolvant le problème suivant, j'ai défini ces 2 tableaux :

```
alpha = "abcdefghijklmnopqrstuvwxyz"
Alpha = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

Au début, je définissais les variables ainsi :

```
tab = []
for k in range(num_reines):
    if (k<=25):
        tab.append(model.new_int_var(0, num_reines-1, alpha[k]))
    else:
        tab.append(model.new_int_var(0, num_reines-1, Alpha[k-26]))
```

Cependant, cela limite le nombre de variables possibles à 52. Or, pour les problèmes avec un nombre de variables pas aussi définis que pour les 2 problèmes précédents, l'idéal serait de pouvoir tester le nombre le plus élevé de variables possibles.

Avec cette librairie, pour que le solveur cherche plusieurs solutions possibles au lieu d'une seule, il faut définir une classe servant à énumérer les solutions possibles. Cette classe prend comme paramètre le problème et a souvent les 3 fonctions : "_init_" pour initialiser le comptage de solutions ; "on_solution_callback" pour compter les solutions et les afficher ; et "solution_count" pour renvoyer le nombre de solutions comptées. Dans la fonction "_init_" est utilisée une autre fonction nommée "CpSolverSolutionCallback.__init__", qui permet de débiter le comptage du nombre de solutions possibles.

Au début, pour calculer toutes les solutions possibles, cette classe a été définie ainsi :

```
class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):

    def __init__(self, variables: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0

    def on_solution_callback(self) -> None:
        self.__solution_count += 1
        for v in self.__variables:
            print(f"{v}={self.value(v)}", end=" ")
        print()

    @property
    def solution_count(self) -> int:
        return self.__solution_count
```

Peu importe le nombre de solutions à trouver, le solveur est toujours initialisé ainsi :

```
solver = cp_model.CpSolver()
```

Si on veut permettre l'énumération des solutions, alors on écrit ces lignes :

```
solution_printer = VarArraySolutionPrinter(tab)
solver.parameters.enumerate_all_solutions = True
```

Puis le programme cherche les solutions possibles et en affiche le nombre et le temps de recherche :

```
debut = time.perf_counter()
status = solver.solve(model, solution_printer)
fin = time.perf_counter()

print()
print("Duree_de_la_recherche_de_solutions=", (fin-debut), "s")
print(f"Statut={solver.status_name(status)}")
print(f"Nombre_de_solutions_trouvees={solution_printer.solution_count}")
```

Pour la signification de la valeur du statut du solveur, voici un tableau préparé par les développeurs d'OR-Tools [20] :

État	Description
OPTIMAL	Une solution réalisable optimale a été trouvée.
FEASIBLE	Une solution réalisable a été trouvée, mais nous ne savons pas si elle est optimale.
INFEASIBLE	Le problème s'est avéré irréalisable.
MODEL_INVALID	Le CpModelProto donné n'a pas réussi l'étape de validation. Vous pouvez obtenir plus détaillée en appelant <code>ValidateCpModel(model_proto)</code> .
UNKNOWN	L'état du modèle est inconnu, car aucune solution n'a été trouvée n'a pas été prouvée INFEASIBLE) avant que le résolveur ne réussisse un arrêt, comme une limite de temps , une limite de mémoire ou une limite personnalisée définie par l'utilisateur.

FIGURE 9 – Valeurs possibles du statut du solveur

Pour 4 reines, 2 solutions ont été trouvées en $6,31.10^{-3}$ secondes. Avec 10 reines, la recherche d'une solution a duré 77 secondes. Malheureusement, j'ai oublié d'écrire le nombre de solutions trouvées dans ce cas-là et je n'arrive plus à le retrouver. Techniquement, il est possible de mettre 27 reines, mais la recherche des solutions a duré tellement longtemps que je l'ai interrompue avant la fin.

Suite à ça, j'ai décidé que mon programme ne chercherait qu'une seule solution, donc qu'il arrête sa recherche à la première solution trouvée. J'ai donc initialisé le solveur et commencé la recherche de la solution ainsi :

```
solver = cp_model.CpSolver()
debut = time.perf_counter()
status = solver.solve(model)
fin = time.perf_counter()
```

Avec 10 reines, la recherche d'une solution a duré $3,77.10^{-2}$ secondes. Avec 300 reines, la recherche d'une solution a duré 796 secondes.

Dès 674 reines, l'exécution du programme s'arrêtait anormalement sans retourner de solution. Au lieu de m'afficher une solution ou "Pas de solution trouvée.", l'invite de commande m'affichait "Processus arrêté". La raison de ce fait est sans doute qu'avec un nombre de reines aux alentours de 674, le programme consommait trop de mémoire vive de l'ordinateur. Il a donc été envisagé que j'utilise la fonction "getrusage" pour vérifier la consommation de mémoire de ce programme et d'autres également. Cependant, j'avais des difficultés à comprendre le fonctionnement de "getrusage" et la façon de l'utiliser, puis je me suis concentrée sur le problème des infirmiers, donc je ne me suis pas penchée sur cet aspect des solveurs.

5.4 Problème d'horaire des infirmiers

Ici, ce problème consiste à attribuer des plages de travail aux infirmiers selon le nombre de jours, de services par jour et d'infirmiers ainsi que d'autres contraintes. Il existe plusieurs variantes de ce problème. Voici les contraintes les plus courantes : pour chaque plage horaire, il faut qu'au moins un infirmier la travaille ; et pour chaque infirmier, il faut qu'il travaille au moins une plage horaire.

Pour ce problème, j'ai récupéré et adapté le code du site web d'OR-Tools. Le programme dont je vais vous parler se trouve ici, en plus des annexes.

Pour me pencher sur ce problème, je me suis basée sur l'exercice 5 du TD d'Algorithmie Avancée sur la programmation linéaire de nombres entiers. Ici, il faut planifier les services de 3 infirmiers (A, B, C) sur 7 jours, sachant qu'il y a 1 plage horaire de travail (appelé "shift" ou "service" dans le code) par jour et que chaque plage horaire doit être assignée à au moins un infirmier. Dans cet exercice de TD, il est aussi défini pour chaque infirmier un nombre minimal et maximal de jours consécutifs de travail ainsi qu'un nombre minimal de jours de repos. Un infirmier peut aussi avoir des jours fixes de repos. Dans la variante que je traite, voici les contraintes suivantes sur les infirmiers :

Infirmier	Min jours consécutifs	Max jours consécutifs	Min jours repos	Jours fixes de repos
A	3	5	2	
B	2	3	2	5
C	2	4	3	0, 5

Ici, le jour 0 désigne le premier jour et le jour 5 le sixième jour.

Le code d'OR-Tools définit déjà les données de base, même si avec des valeurs différentes. Soient `num_nurses` le nombre d'infirmiers, `num_days` le nombre de jours et `num_shifts` le nombre de plages horaire par jour. Dans le code, il est défini un dictionnaire nommé "shifts" de dimensions $*num_nurses * num_days * num_shifts$, avec des variables booléennes déterminant si un infirmier travaille une plage horaire donnée durant un certain jour. Comme montré par la suite, je peux utiliser dessus la fonction "sum" sans problème. Pourtant, cette fonction ne fonctionne que sur des tableaux avec uniquement des valeurs numériques. Je suppose alors que les variables de "shifts" sont en réalité des entiers entre 0 et 1.

Pour un premier test, j'en ai juste modifié les valeurs :

```
num_nurses = 3
num_shifts = 1
num_days = 7
all_nurses = range(num_nurses)
all_shifts = range(num_shifts)
all_days = range(num_days)
```

Avec seulement les contraintes définies par OR-Tools, et en modifiant juste les données de base comme plus haut, j'obtiens 630 solutions en $3,19.10^{-2}$ secondes.

J'ai gardé une bonne partie du code du site web d'OR-Tools, mais je voulais l'adapter à cet exercice de TD. Pour cela, j'ai dû enlever des contraintes qui étaient définies dans le code de base. Parmi ces contraintes, il y a celle qu'il n'y a qu'un seul infirmier par plage horaire.

J'ai ensuite défini les contraintes plus haut.

D'abord, j'ai fait en sorte que le programme prenne en compte le nombre minimal de jours de repos. Pour chaque infirmier, j'ai défini un encadrement faisant que son nombre de jours de travail soit inférieur ou égal à la différences entre le nombre total de plages horaires à traiter et son nombre minimal de jours de repos. Ainsi, j'obtiens 130 solutions en $7,52.10^{-3}$ secondes.

Puis j'ai fait en sorte que chaque infirmier se repose durant les jours fixes de repos, s'il en a. Avec cette contrainte supplémentaire, j'obtiens 30 solutions en $3,65.10^{-3}$ secondes.

Enfin, je voulais que le nombre de jours consécutifs soient pris en compte. Mais j'avais du mal à m'occuper des jours consécutifs maximaux, donc je ne me suis penchée que sur les minimaux.

J'ai rajouté un boucle for un peu longue avec dedans un ensemble de boucles while pour que le nombre minimal de jours consécutifs de travail de l'infirmier soit respecté. En effet, il faut vérifier si ce minimum peut être respecté à partir

d'un certain jour, en prenant en compte le nombre total de jours et les jours fixes de repos.

Ainsi j'obtiens 30 solutions en $1,25 \cdot 10^{-2}$ secondes. On peut remarquer que j'obtiens le même nombre de solutions que lorsque j'ai fait en sorte que les jours fixes soient pris en compte, mais que le temps de recherche des solutions a augmenté. D'ailleurs, pour ce programme, j'ai fait en sorte qu'il cherche toutes les solutions et m'en donne le nombre, mais après ne m'affiche qu'une seule solution. Par conséquent, je ne peux pas vérifier s'il s'agit des mêmes solutions.

Des programmes abordés dans le rapport, celui-ci me satisfait le moins. Il est inachevé. Il y reste encore des corrections à faire, mais j'en ai difficilement eu le temps avant de me pencher sur la conception d'une interface tangible. En le relisant, je trouve aussi qu'il a trop de lignes pour rien. Il y a sans doute une manière de faire un programme plus fonctionnel avec un temps de traitement plus court.

Pour revenir au problème des infirmiers, nos encadrants et partenaires m'ont demandé d'imaginer une interface tangible pour le résoudre et en fasse une maquette.

6 Maquettes d'interface tangible

Après m'être penchée sur la manière coder des solveurs pour ces problèmes d'attribution, j'ai réfléchi à quelle interface tangible pourrait aider à résoudre le problème de répartition du service des infirmiers. Ici, le mot "maquette" désigne un schéma permettant d'illustrer le fonctionnement de cette interface. J'ai fait les schémas ci-dessous sur Canva.

Pour rappel, voici les contraintes courantes de ce problème :

- chaque plage horaire doit être travaillée par au moins un infirmier ;
- chaque infirmier doit travailler au moins une plage horaire ;
- après un nombre plus ou moins élevé de services consécutifs, l'infirmier doit pouvoir se reposer.

Selon les variantes du problème, il existe bien entendu d'autres contraintes. Notamment que, si un infirmier fait un service de nuit, alors il ne doit pas travailler le lendemain pendant la journée. Aussi, les infirmiers peuvent avoir exprimé leurs préférences, notamment celle de travailler le matin. Cependant, je préfère rester au plus simple pour la conception de cette interface tangible.

Je me suis basée sur les données suivantes :

- 3 infirmiers ;
- 7 jours à traiter ;
- 1 plage horaire de travail par jour.

L'interface que je vais proposer permet de choisir dans une zone de sélection jusqu'à 6 semaines. Voici mes raisons de choisir 6 comme nombre maximal de semaines à traiter. La première est qu'il serait préférable que l'utilisateur puisse choisir le nombre de semaines à traiter d'une manière tangible, et donc qu'il faut gérer l'espace sur la machine. Le nombre de six est à la fois assez élevé pour permettre de traiter la répartition des services aux infirmiers sur une assez

longue période (d'un mois et demi) et assez bas pour ne pas prendre trop de place sur l'interface tangible. La deuxième est de permettre une plus grande flexibilité sur le nombre de services par jour. Imaginons que l'utilisateur choisisse 6 semaines. Si on reste sur 1 service par jour, alors l'utilisateur traite 6 semaines donc 42 jours. Par contre, si l'utilisateur souhaite 2 services par jour, alors ce qu'il fait équivaut à traiter 3 semaines donc 21 jours, car $42/2 = 21$. Aussi, si l'utilisateur a en tête 3 services par jour, alors ce qu'il fait équivaut à traiter 2 semaines donc 14 jours, car $42/3 = 14$.

Enfin, voici 2 principales possibilités que je vois pour représenter les choix possibles de nombre de semaines : soit des boutons sur lesquels appuyer pour choisir un certain nombre de semaines, soit des petites zones de sélection sur lesquelles mettre un jeton. Pour plus de clarté dans les schémas, je vais utiliser l'option des petites zones de sélection et du jeton.

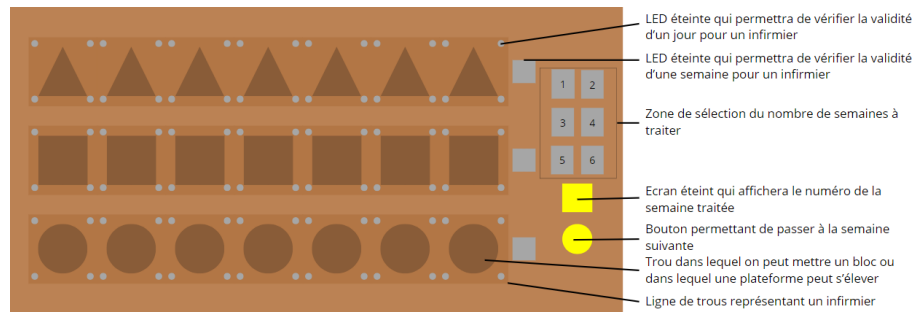


FIGURE 10 – Maquette du TUI pour le problème d'infirmiers dans son état initial

Cette interface tangible est inspirée de jouets de tri de formes. Ce jouet pour enfant est souvent une grande boîte ou un grand cube avec des trous de formes précises (triangle, carré, rond...), associé à des blocs faits d'une de ces formes. Le but est de mettre ces blocs dans la boîte ou le cube en les faisant passer via le bon trou.

Pour cette interface tangible (Figure 10), nous avons un peu le même principe. Pour traiter une semaine, l'utilisateur doit mettre au maximum 4 blocs par infirmier dans un des trous de la ligne correspondante. Ces blocs correspondent au début ou à la fin d'une période de travail pour l'infirmier. L'utilisateur n'est pas obligé de placer un bloc pour arrêter une période de travail. Cependant, il est obligé d'en mettre un s'il veut commencer une période de travail pour l'infirmier. (Figure 12)

Dans les trous se trouvent des plateformes que l'interface tangible peut élever selon les jours de travail et de repos déjà décidés. En effet, les plateformes s'élèvent pour représenter les jours de repos et pour couvrir le nombre minimal de jours de travail de l'infirmier. (Figures 11 et 12) Les jours de repos fixes et le nombre minimal et maximal de jours consécutifs par infirmier sont donnés par l'utilisateur en configurant le système de l'interface tangible. Aussi, l'interface

tangible met par défaut un jour de repos juste avant une période de travail et un autre jour de repos juste après le nombre maximal de jours de travail de l'infirmier. (Figure 12)

Comme montré dans la figure suivante, l'utilisateur a mis le jeton sur la zone représentant 1, ce qui lui permet de ne traiter qu'1 semaine. Par la suite, il peut configurer le système de l'interface tangible.

La façon de configurer l'interface tangible pour l'adapter à chaque instance d'un même problème voire au traitement d'un problème différent est un vaste sujet, qui demanderait une étude plus approfondie. Pour cette raison, je vais juste parler des données que j'imagine l'utilisateur mettre dans le système de cette interface tangible, dans la situation que je vais vous illustrer. Je vais encore une fois me baser sur l'instance suivante :

Infirmier	Min jours consécutifs	Max jours consécutifs	Min jours re-pos	Jours fixes de repos
A	3	5	2	
B	2	3	2	5
C	2	4	3	0, 5

Partons du principe que les jours d'une semaine sont numérotés de 0 à 6.

Le premier infirmier peut effectuer une période de travail d'au minimum 3 jours et d'au maximum 5 jours. Il n'a aucun jour de repos fixe.

Le deuxième infirmier peut effectuer une période de travail d'au minimum 2 jours et d'au maximum 3 jours. Il a comme jour de repos fixe le jour n°5.

Le troisième infirmier peut effectuer une période de travail d'au minimum 2 jours et d'au maximum 4 jours. Il a comme jours de repos fixes les jours n°0 et n°5.

Enfin, l'utilisateur peut mettre dans le système le nombre minimal de jours de travail par infirmier et par semaine. Imaginons qu'il mette comme nombre 4.

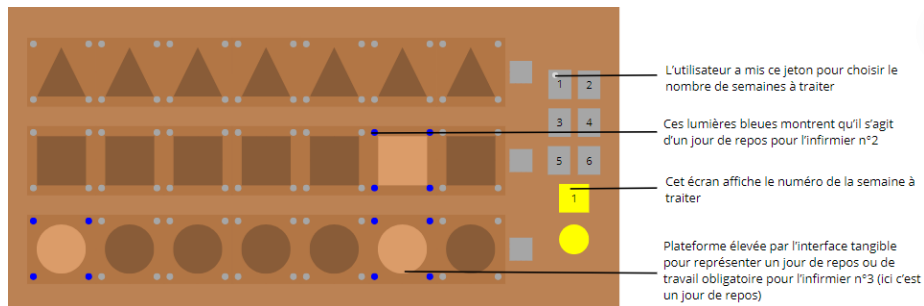


FIGURE 11 – Maquette du TUI après configuration

Comme montré dans la figure ci-dessus, pour chaque infirmier, les plate-formes (beiges) se sont élevées dans les trous représentant ses jours fixes de repos.

Pour les petits LED autour des trous, blocs ou plateformes, ils ont plusieurs significations possible selon leur couleur :

- en gris : ils ne sont simplement pas allumés. Tant qu'il n'y a rien dans ces trous, les jours qu'ils représentent sont par défaut considérés comme des jours de repos ;
- en vert : le trou, le bloc ou la plateforme qu'ils entourent représente un jour de travail (certains pour un bloc ou une plateforme) ;
- en bleu, la plateforme représente un jour de repos ;
- en rouge, ils signalent une erreur.

Imaginons ensuite que l'utilisateur pose les blocs de la manière ci-dessous :

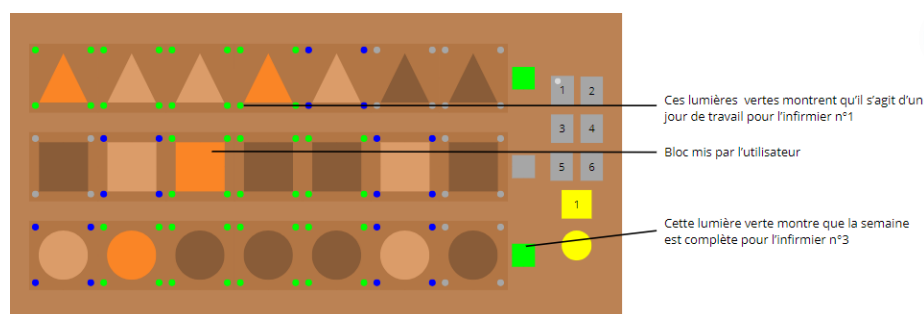


FIGURE 12 – Maquette du TUI après la pose des premiers blocs

Comme montré ci-dessus, l'utilisateur a posé les blocs (orange) ainsi. Quand un bloc débute une période de travail, s'il y a un trou à gauche, alors une plateforme s'élève dedans pour le boucher, montrant ainsi une période de repos. Puis les jours suivants sont comptés par le système comme étant des jours de travail jusqu'à ce qu'on atteigne un jour de repos déjà établi ou le nombre maximal de jours consécutifs pour l'infirmier correspondant. Par la suite, s'il n'y a pas déjà de jour de repos et s'il y a un trou à droite de ceux représentant les jours de travail, alors une plateforme s'élève dedans pour le boucher, montrant ainsi une période de repos. De plus, les plateformes s'élèvent jusqu'au jour précédant celui du nombre minimal de jours consécutifs, afin de montrer des jours certains de travail. L'utilisateur peut mettre un bloc dans un des trous représentant les autres jours de cette période de travail. Ainsi, il conclut la période de travail et les trous précédant ce dernier bloc se font boucher par leurs plateformes. D'ailleurs, c'est ce qui s'est passé pour le premier infirmier.

Quant aux gros LED à côté des lignes, ils permettent de vérifier la complétion d'une semaine pour un infirmier. Eux aussi ont des significations possibles selon leur couleur :

- en gris : ils ne sont simplement pas allumés. Ici, cela signifie que la semaine est incomplète pour l'infirmier, mais sans qu'il n'y ait des erreurs ;
- en vert : la semaine est complète pour l'infirmier ;
- en rouge : il y a une ou plusieurs erreurs pour cet infirmier.

Ici, la semaine est complète pour les premier et troisième infirmiers, mais

pas pour le deuxième. En effet, contrairement aux autres infirmiers, le deuxième effectue moins de 4 jours de travail.

Les schémas ci-dessus ont été faits sur Canva. Le logigramme disponible en 2 parties dans les annexes donne un meilleur aperçu du fonctionnement de cette interface tangible que j'ai imaginée. (Figures 13 et 14)

7 Conclusion

7.1 Bilan du travail effectué

Durant le mois de mai, Adrien et moi avons fait nos recherches sur les interfaces tangibles et les problèmes de contraintes. A partir du fruit de nos recherches, nous avons chacun écrit un état de l'art sur les interfaces tangibles, les problèmes de satisfaction de contraintes et leur rôle dans notre projet. Nos états de l'art pourraient servir de référence pour de futurs stagiaires et doctorants. Ils peuvent notamment s'aider de l'état de l'art et apporter de nouvelles informations pouvant le corriger. Ensuite, nous avons écrit des programmes pour résoudre des problèmes de contraintes. Enfin, nous avons imaginé une interface tangible pour résoudre un problème de contraintes qui nous avait été attribué avant de préparer un ensemble de maquettes illustrant son fonctionnement.

Mes travaux divergent de ceux de mon co-stagiaire sur plusieurs aspects, notamment concernant le développement des programmes visant à résoudre les problèmes de contraintes. Ainsi, je me suis davantage concentrée sur les problèmes d'attribution et me suis tout particulièrement intéressée à la variété des problèmes et la manière dont le système peut les résoudre.

7.2 Bilan personnel

Ce stage m'a permis d'appliquer nos compétences acquises jusqu'en M1, d'approfondir mes connaissances en informatique et de mieux comprendre les problèmes de contraintes. J'ai aussi renforcé les compétences suivantes : celle de la collaboration avec une équipe de projet et celle de la lecture d'articles scientifiques. De plus, j'ai découvert l'organisation au sein d'un institut de recherche ainsi que le quotidien des chercheurs.

En général, je peux avoir des difficultés avec mon organisation personnelle. Cependant, ce stage m'a aidée à progresser sur ce point. De plus, ma volonté de bien faire m'a poussée à m'investir et à réaliser le travail le plus fourni possible. Ainsi, j'ai pu faire une grande majorité de ce qui m'a été demandé. Enfin, j'ai constaté qu'il est courant dans la recherche d'explorer une nouvelle piste en tâtonnant, sans certitude de ne pas tomber sur un cul-de-sac. J'ai donc appris à appréhender cet aspect du métier de chercheur.

7.3 Perspectives

D'après ce que j'ai compris de la suite du projet, il y aura de futurs stages et thèses sur le sujet. Notamment, un doctorant se penchera sur le projet dès le 1er novembre. Les objectifs futurs seront de déchiffrer, creuser et imaginer les besoins sur les contraintes, afin de concevoir une interface tangible pour plusieurs personnes colocalisées.

Une possibilité pour l'avancée du projet est que notre classification des problèmes de contraintes soit affinée. En plus de régler les ambiguïtés mentionnées

dans le rapport, ses modifications pourraient permettre de classer les problèmes en fonction des lois physiques pouvant représenter leurs contraintes.

Une question qui sera bientôt abordée dans le projet est la suivante : est-ce qu'une interface tangible générique pouvant résoudre tous les problèmes de contraintes pourraient exister ? Selon Florence Bannay, c'est possible. Durant une réunion, elle a parlé d'une interface tangible qu'elle s'est imaginée dans ce but. Pour Adrien et moi, c'est plus probable qu'il y en ait bientôt plusieurs qui pourraient résoudre plusieurs problèmes avec des contraintes similaires. L'affinement de notre classification de contraintes pourrait aider à la conception de ces interfaces tangibles.

8 Bibliographie

- [1] Sean FOLLMER et al. “inFORM : dynamic physical affordances and constraints through shape and object actuation”. In : *Proceedings of the 26th annual ACM symposium on User interface software and technology*. UIST’13 : The 26th Annual ACM Symposium on User Interface Software and Technology. St. Andrews Scotland, United Kingdom : ACM, 8 oct. 2013, p. 417-426. ISBN : 978-1-4503-2268-3. DOI : 10.1145/2501988.2502032.
- [2] Khaled GHEDIRA. *Problèmes de satisfaction de contraintes : formalismes et techniques CSP*. Collection Informatique. Paris : Hermès science publications-Lavoisier, 2013. ISBN : 978-2-7462-4515-0.
- [3] James HINES et al. “Construction by replacement : a new approach to simulation modeling”. In : *System Dynamics Review* 27.1 (jan. 2011), p. 64-90. ISSN : 0883-7066, 1099-1727. DOI : 10.1002/sdr.437.
- [4] Eva HORNECKER. “Understanding the Benefits of Graspable Interfaces for Cooperative Use”. In : *Cooperative Systems Design : A Challenge of the Mobility Age*. IOS Press. 2002, p. 71-87. ISBN : 978-1-58603-244-9.
- [5] Eva HORNECKER et Jacob BUUR. “Getting a grip on tangible interaction : a framework on physical space and social interaction”. In : *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI ’06. New York, NY, USA : Association for Computing Machinery, 22 avr. 2006, p. 437-446. ISBN : 978-1-59593-372-0. DOI : 10.1145/1124772.1124838.
- [6] Hiroshi ISHII et Brygg ULLMER. “Tangible bits : towards seamless interfaces between people, bits and atoms”. In : *Proceedings of the ACM SIGCHI Conference on Human factors in computing systems*. CHI97 : ACM Conference on Human Factors & Computing Systems. Atlanta Georgia USA : ACM, 27 mars 1997, p. 234-241. ISBN : 978-0-89791-802-2. DOI : 10.1145/258549.258715.
- [7] Hiroshi ISHII et al. “Radical atoms : beyond tangible bits, toward transformable materials”. In : *Interactions* 19.1 (jan. 2012), p. 38-51. ISSN : 1072-5520, 1558-3449. DOI : 10.1145/2065327.2065337.
- [8] Hans-Christian JETTER et al. “Does (multi-)touch aid users’ spatial memory and navigation in ‘panning’ and in ‘zooming & panning’ UIs?” In : *Proceedings of the International Working Conference on Advanced Visual Interfaces*. AVI ’12. New York, NY, USA : Association for Computing Machinery, 21 mai 2012, p. 83-90. ISBN : 978-1-4503-1287-5. DOI : 10.1145/2254556.2254575.
- [9] David KIRSH et Paul MAGLIO. “On Distinguishing Epistemic from Pragmatic Action”. In : *Cognitive Science* 18.4 (oct. 1994), p. 513-549. ISSN : 0364-0213, 1551-6709. DOI : 10.1207/s15516709cog1804_1.

- [10] Scott R. KLEMMER, Björn HARTMANN et Leila TAKAYAMA. “How bodies matter : five themes for interaction design”. In : *Proceedings of the 6th conference on Designing Interactive systems*. DIS '06. New York, NY, USA : Association for Computing Machinery, 26 juin 2006, p. 140-149. ISBN : 978-1-59593-367-6. DOI : 10.1145/1142405.1142429.
- [11] LEONHARD EULER. *E309–Solution d’une question curieuse qui ne paroit soumise à aucune analyse*. 1766. 28 p. URL : <http://archive.org/details/euler-e309>.
- [12] Yehuda NAVEH. “The Big Deal : Applying Constraint Satisfaction Technologies Where It Makes the Difference”. In : *Theory and Applications of Satisfiability Testing – SAT 2010*. Sous la dir. d’Ofer STRICHMAN et Stefan SZEIDER. T. 6175. Series Title : Lecture Notes in Computer Science. Berlin, Heidelberg : Springer Berlin Heidelberg, 2010, p. 1-7. ISBN : 978-3-642-14185-0 978-3-642-14186-7. DOI : 10.1007/978-3-642-14186-7_1.
- [13] *Nurse scheduling problem*. In : *Wikipedia*. Page Version ID : 1219712085. 19 avr. 2024. URL : https://en.wikipedia.org/w/index.php?title=Nurse_scheduling_problem&oldid=1219712085.
- [14] James PATTEN, Ben RECHT et Hiroshi ISHII. “Audiopad : a tag-based interface for musical performance”. In : *Proceedings of the 2002 conference on New interfaces for musical expression*. NIME '02. SGP : National University of Singapore, 1^{er} mai 2002, p. 1-6. ISBN : 978-1-874653-65-3.
- [15] James PATTEN et al. “Sensetable : a wireless object tracking platform for tangible user interfaces”. In : *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '01. New York, NY, USA : Association for Computing Machinery, 1^{er} mars 2001, p. 253-260. ISBN : 978-1-58113-327-1. DOI : 10.1145/365024.365112.
- [16] *Présentation - IRIT*. URL : <https://www.irit.fr/le-laboratoire/presentation-du-laboratoire/>.
- [17] Stéphanie REY. “Apports des interactions tangibles pour la création, le choix et le suivi de parcours de visite personnalisés dans les musées”. Thèse de doct. Université de Bordeaux, 8 juin 2020. URL : <https://theses.hal.science/tel-02929283>.
- [18] Philipp SCHOESSLER et al. “Kinetic Blocks : Actuated Constructive Assembly for Interaction and Display”. In : *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*. UIST '15 : The 28th Annual ACM Symposium on User Interface Software and Technology. Charlotte NC USA : ACM, 5 nov. 2015, p. 341-349. ISBN : 978-1-4503-3779-3. DOI : 10.1145/2807442.2807453.
- [19] Orit SHAER et Eva HORNECKER. “Tangible User Interfaces : Past, Present, and Future Directions”. In : *Found. Trends Hum.-Comput. Interact.* 3.1 (1^{er} jan. 2010), p. 1-137. ISSN : 1551-3955. DOI : 10.1561/11000000026.
- [20] *Solveur CP-SAT / OR-Tools*. Google for Developers. URL : https://developers.google.com/optimization/cp/cp_solver?hl=fr.

- [21] *Sujets de stage M2 ou Thèses ANNEE 2020*. URL : https://www.irit.fr/~Florence.Bannay/M2_SUJETSTAGES_2024.html.
- [22] B. ULLMER et H. ISHII. “Emerging frameworks for tangible user interfaces”. In : *IBM Systems Journal* 39.3 (2000), p. 915-931. ISSN : 0018-8670. DOI : 10.1147/sj.393.0915.
- [23] *XCSP3*. XCSP3. URL : <http://xcsp.org/>.

9 Annexes

9.1 Codes de résolution des problèmes de contraintes

9.1.1 Ordonnancement des tâches

```
from constraint import * #importation de la librairie python-constraint
import time

problem = Problem() #initialisation du problème
problem.addVariables("A", range(7)) #initialisation de la première tâche
problem.addVariables("B", range(7)) #initialisation de la deuxième tâche
problem.addVariables("C", range(7)) #initialisation de la troisième tâche
problem.addVariables("D", range(7)) #initialisation de la quatrième tâche

problem.addConstraint(lambda C, D: C + 3 <= D, ("C", "D")) #la tâche 3 (de durée 3) doit se faire
#avant la tâche 4
problem.addConstraint(lambda B, C: (B <= C - 1)|(C + 3 <= B), ("B", "C")) #les tâches 2 (de durée 1)
#et 3 (de durée 3) ne peuvent se faire en parallèle

#fonctions ajoutées pour minimiser la date maximale de fin d'une tâche et faire en sorte que les
#tâches se finissent en maximum 7 secondes

def fin_max(a,b,c,d): #fonction qui donne la date maximale de fin d'une tâche
    f1 = a + 5 #car la 1ère tâche a comme durée d1 = 5
    f2 = b + 1 #car la 2ème tâche a comme durée d2 = 1
    f3 = c + 3 #car la 3ème tâche a comme durée d3 = 3
    f4 = d + 4 #car la 4ème tâche a comme durée d4 = 4
    return max(f1,f2,f3,f4)

def min_fin_max(a,b,c,d): #fonction permettant de déterminer si la date maximale de fin d'une tâche
#est inférieure ou égale à 7
    return (fin_max(a,b,c,d) <= 7)

problem.addConstraint(lambda A,B,C,D: min_fin_max(A,B,C,D), ("A", "B", "C", "D")) #ajout de la
#contrainte de minimisation de la durée max de fin de tâche

debut = time.perf_counter() #debut de la recherche de solutions
solutions = problem.getSolutions() #résolution du problème
fin = time.perf_counter() #fin de la recherche de solutions
print("Durée_de_la_recherche_de_solutions:", (fin-debut), "s") #affichage de la durée de la recherche
#de solutions

print("Nombre_de_solutions:", len(solutions), "\n")

for i in range(len(solutions)): #affichage des solutions
    print("Solution_n°", i+1, ":") #affichage du numéro de la solution
    print("debut_de_la_tache_A:", solutions[i]["A"]) #affichage du début de la tâche A dans cette
#solution
    print("debut_de_la_tache_B:", solutions[i]["B"]) #affichage du début de la tâche B dans cette
#solution
    print("debut_de_la_tache_C:", solutions[i]["C"]) #affichage du début de la tâche C dans cette
#solution
    print("debut_de_la_tache_D:", solutions[i]["D"]) #affichage du début de la tâche D dans cette
#solution
    if (i < len(solutions)-1):
        print() #si on n'est pas à la dernière solution, sauter une ligne
```

9.1.2 Répartition des employés de cinéma

```
from constraint import * #importation de la librairie python-constraint
import time

employees = ["Andrew", "David", "Jane", "Jason", "Leslie", "Michael", "Marilyn", "Oliver"] #liste des
#employés
#A partir de maintenant, on aura : Andrew = 0, David = 1, Jane = 2, Jason = 3, Leslie = 4,
#Michael = 5, Marilyn = 6 et Oliver = 7
locations = ["le_guichet", "la_premiere_entree", "la_seconde_entree", "les_vestiaires"] #liste des
#locations
locations2 = [["a", "b", "c"], ["d", "e"], ["f", "g"], ["h"]] #liste des places possibles dans ces
#locations, elles doivent toutes être occupées, ces places serviront de variables

problem = Problem() #initialisation du problème

#initialisation des variables
for i in range(len(locations2)):
    for j in range(len(locations2[i])):
        problem.addVariables(locations2[i][j], range(8)) #each location must be occupied by exactly the
#number of employees it requires;

#initialisation des contraintes
problem.addConstraint(AllDifferentConstraint()) #each employee must have a unique location
problem.addConstraint(lambda f, g: f==4 or g==4, ("f", "g")) #Leslie must work at the second entrance
problem.addConstraint(lambda d, e: d==5 or e==5, ("d", "e")) #Michael must work at the first entrance;
for i in range(len(locations2)-1):
    for j in range(len(locations2[i])-1):
        for k in range(j+1, len(locations2[i])):
            #David, Michael and Jason cannot work together;
            problem.addConstraint(lambda l, m: l!=1 or m!=5, (locations2[i][j], locations2[i][k]))
            problem.addConstraint(lambda l, m: l!=3 or m!=5, (locations2[i][j], locations2[i][k]))
            problem.addConstraint(lambda l, m: l!=1 or m!=3, (locations2[i][j], locations2[i][k]))
            #Pour essayer de régler le 1er problème de symétrie constaté
            problem.addConstraint(lambda l, m: l<m, (locations2[i][j], locations2[i][k]))
            if (i==0): #if David is selling tickets, Marilyn must be with him
                problem.addConstraint(lambda l, m: l!=1 or m==6, (locations2[i][j], locations2[i][k]))

debut = time.perf_counter() #debut de la recherche de solutions
solutions = problem.getSolutions() #résolution du problème
fin = time.perf_counter() #fin de la recherche de solutions
print("Durée de la recherche de solutions:", (fin-debut), "s") #affichage de la durée de la recherche
#de solutions

def affichage(num): #fonction pour afficher les employés présents à chaque location dans la
#solution n°num
    ch = ""
    for ind in range(4):
        ch += "Dans_ " + locations[ind] + " :_ "
        for i in range(len(locations2[ind])):
            if ((len(locations2[ind])>1)&(i>0)):
                if (i==len(locations2[ind])-1):
                    ch += "et_"
                else:
                    ch += ",_"
            ch += affichage2(solutions[num][locations2[ind][i]])
    if (ind<3):
```

```

        ch += "\n"
    return ch

def affichage2(ind): #fonction pour afficher l'employé associé à l'entier ind
    ch = ""
    if (0 <= ind <= 7):
        ch += employees[ind]
    return ch

print("Nombre de solutions: ", len(solutions), "\n")
#affichage des solutions
for i in range(len(solutions)):
    print("Solution", i+1, ":")
    print(affichage(i))
    if (i<len(solutions)-1):
        print()

```

9.1.3 n-reines

Plusieurs solutions

```
import time, collections
from ortools.sat.python import cp_model

num_reines = int(input("Nombre de reines: "))

alpha = "abcdefghijklmnopqrstuvwxyz"
Alpha = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, variables: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0

    def on_solution_callback(self) -> None:
        self.__solution_count += 1
        for v in self.__variables:
            print(f"{v}={self.value(v)}", end=" ")
        print()

    @property
    def solution_count(self) -> int:
        return self.__solution_count

def search_for_all_solutions_sample_sat():
    """Showcases calling the solver to search for all solutions."""
    # Creates the model.
    model = cp_model.CpModel()

    tab = [] #ensemble des variables représentant une reine à chaque ligne de l'échiquier

    for k in range(num_reines):
        if (k<=25):
            tab.append(model.new_int_var(0, num_reines-1, alpha[k]))
        else:
            tab.append(model.new_int_var(0, num_reines-1, Alpha[k-26]))

    for i in range(len(tab)):
        for j in range(i+1, len(tab)):
            model.add(tab[i] != tab[j]) #2 reines ne doivent pas être à la même colonne
            #Note : chaque variable représente une ligne dans laquelle se trouve une reine
            #donc pas besoin de mettre une contrainte alldiff pour les lignes
            if (abs(i-j)==1): #les colonnes de 2 reines ne doivent pas être côte à
côte
                model.add(tab[i] != (tab[j]-1))
                model.add(tab[i] != (tab[j]+1))

    # Create a solver and solve.
    solver = cp_model.CpSolver()
    solution_printer = VarArraySolutionPrinter(tab)
    # Enumerate all solutions.
    solver.parameters.enumerate_all_solutions = True
```

```

# Solve.
debut = time.perf_counter() #debut de la recherche de solutions
status = solver.solve(model, solution_printer)
fin = time.perf_counter() #fin de la recherche de solutions

print()
print("Duree_de_la_recherche_de_solutions=", (fin-debut), "s")
print(f"Statut={solver.status_name(status)}")
print(f"Nombre_de_solutions_trouvees:{solution_printer.solution_count}")

search_for_all_solutions_sample_sat()

```

Une solution

```
import time, collections
from ortools.sat.python import cp_model

#ces deux listes me permettent de nommer les variables
alpha = "abcdefghijklmnopqrstuvwxyz"
Alpha = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

num_reines = int(input("Nombre de reines: ")) #l'utilisateur doit saisir le nombre de reines
print()

model = cp_model.CpModel() #création du problème

tab = []
for k in range(num_reines):
    #une manière de nommer les variables permettant en théorie de mettre le nombre de variables
    #qu'on veut
    name = ""
    n = k//26
    tab2 = [n]
    while(n>25):
        n = n//26
        tab2.append(n)
    name = name + alpha[tab2[len(tab2)-1]]
    for l in range(len(tab2)-1):
        name = name + alpha[tab2[len(tab2)-1-l]//(26*(len(tab2)-1-l))]
    name = Alpha[k%26] + name
    tab.append(model.new_int_var(1, num_reines, name))

#initialisation des contraintes
for i in range(num_reines-1):
    for j in range(i+1, num_reines):
        model.add(tab[i]!=tab[j]) #deux reines ne peuvent pas être dans la même colonne
        model.add(tab[i]+j!=tab[j]+i) #deux reines ne peuvent pas être dans la même diagonale gauche
        model.add(tab[i]-j!=tab[j]-i) #deux reines ne peuvent pas être dans la même diagonale droite
#Note : deux reines ne peuvent pas être dans la même ligne, mais ici chaque reine représente une
#ligne différente, donc pas besoin d'implémenter cette contrainte

solver = cp_model.CpSolver() #création d'un solveur du problème
debut = time.perf_counter() #debut de la recherche d'une solution
status = solver.solve(model) #recherche d'une solution
fin = time.perf_counter() #fin de la recherche d'une solution

def affichage_echiquier(): #affichage des coordonnées des reines
    ch = "Affichage des coordonnées (x,y) des reines telles que x<=y, y<=x" + str(num_reines)
    + "\n"
    for i in range(num_reines):
        ch += "Reine n°" + str(i+1) + ": (" + str(i+1) + ", " + str(solver.value(tab[i])) + ")\n"
    return ch

if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE: #s'il existe une solution
    print(affichage_echiquier())
else: #sinon
    print("Pas de solution.")

print("Durée de la recherche d'une solution: ", (fin-debut), "s") #affichage de la durée de recherche
#d'une solution
```

9.1.4 Problème d'horaire des infirmiers

```
from ortools.sat.python import cp_model
import time

def main() -> None:
    # Data.
    num_nurses = 3
    num_shifts = 1
    num_days = 7
    all_nurses = range(num_nurses)
    all_shifts = range(num_shifts)
    all_days = range(num_days)

    print("Nombre d'infirmiers:", num_nurses)
    print("Nombre de jours:", num_days)
    print("Nombre de services par jour:", num_shifts)
    print()

    #implémentation nombres minimal et maximal de jours consécutifs de travail
    nombre_shifts_consecutifs = [[3,5],[2,3],[2,4]]
    """

    """
    #implémentation nombre minimal de jours de repos
    nombre_minimal_repos = [2,2,3]

    #implémentation jours de repos déjà décidés
    services_repos_deja_decides = [
        [
            [],[],[],[],[],[],[]
        ],
        [
            [],[],[],[],[],[0,1,2],[]
        ],
        [
            [0,1,2],[],[],[],[],[0,1,2],[]
        ],
    ]

    # Creates the model.
    model = cp_model.CpModel()

    # Creates shift variables.
    # shifts[(n, d, s)]: nurse 'n' works shift 's' on day 'd'.
    shifts = {}
    for n in all_nurses:
        for d in all_days:
            for s in all_shifts:
                shifts[(n, d, s)] = model.new_bool_var(f"shift_{n}_{d}_{s}")

    # Each nurse works at most one shift per day.
    for n in all_nurses:
        for d in all_days:
            model.add_at_most_one(shifts[(n, d, s)] for s in all_shifts)
```

```

for n in all_nurses:
    shifts_worked = []
    for d in all_days:
        for s in all_shifts:
            shifts_worked.append(shifts[(n, d, s)])
        #Définition d'un encadrement du nombre de quarts de travail des infirmiers
        #selon jour minimal de repos
        model.add(nombre_minimal_repos[n] <= sum(shifts_worked))
        model.add(sum(shifts_worked) <= (num_shifts * num_days) - nombre_minimal_repos[n])

#Les infirmiers ne travaillent pas durant les jours de repos assignés
for n in all_nurses:
    for d in all_days:
        if (len(services_repos_deja_decides[n][d])>0):
            repos = []
            for s in services_repos_deja_decides[n][d]:
                if s in all_shifts:
                    repos.append(shifts[(n, d, s)])
            model.add(sum(repos) == 0)

#initialisation des contraintes
for n in all_nurses: #shifts de travail et de repos
    shifts_travaillées = []
    for d in all_days:
        for s in all_shifts:
            if s in services_repos_deja_decides[n][d]: #si on tombe sur un shift de repos déjà
#décidé
                model.add(shifts[(n, d, s)] == 0)
                shifts_travaillées.append(shifts[(n, d, s)])
    model.add(nombre_shifts_consecutifs[n][0] <= sum(shifts_travaillées))
    model.add(sum(shifts_travaillées) <= nombre_shifts_consecutifs[n][1])
    model.add(nombre_minimal_repos[n] <= num_days*num_shifts - sum(shifts_travaillées))

for d in all_days: #au moins un infirmier par shift
    for s in all_shifts:
        shifts_occupés = []
        for n in all_nurses:
            shifts_occupés.append(shifts[(n, d, s)])
        model.add(1 <= sum(shifts_occupés))

#Les infirmiers ont un nombre minimal et maximal de jours consécutifs
for n in all_nurses:
    m1 = nombre_shifts_consecutifs[n][0]
    m2 = nombre_shifts_consecutifs[n][1]
    d = 0
    s = 0
    while (d < num_days):
        d2 = d
        s2 = s
        if (s == num_shifts - 1):
            s3 = 0
            d3 = d + 1
        else:
            s3 = s + 1
            d3 = d
        D = 0
        shifts_worked = []

```

```

while (d2 < num_days):
    while ((d2 < num_days)&(D < m1)&(s2 not in services_repos_deja_decides[n][d2])):
        shifts_worked.append(shifts[(n, d2, s2)])
        if ((s2 == num_shifts - 1)&(d2 < num_days)):
            s2 = 0
            d2 += 1
        elif (d2 < num_days):
            s2 += 1
            D += 1
    if ((d2 < num_days)&((D == m1)|(s2 in services_repos_deja_decides[n][d2]))):
        if (s2 == num_shifts - 1):
            s3 = 0
            d3 = d2 + 1
        else:
            s3 = s2 + 1
            d3 = d2
        d2 += 1
    if (len(shifts_worked) == m1):
        model.add(sum(shifts_worked) == m1)
        d = d3
        s = s3
    elif (s == len(all_shifts) - 1):
        s = 0
        d += 1
    else:
        s += 1
print()

# Creates the solver and solve.
solver = cp_model.CpSolver()
solver.parameters.linearization_level = 0
# Enumerate all solutions.
solver.parameters.enumerate_all_solutions = True

class NursesPartialSolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, shifts, num_nurses, num_days, num_shifts, limit):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self._shifts = shifts
        self._num_nurses = num_nurses
        self._num_days = num_days
        self._num_shifts = num_shifts
        self._solution_count = 0
        self._solution_limit = limit

    def on_solution_callback(self):
        self._solution_count += 1
        if (self._solution_count <= self._solution_limit):
            print(f"Solution_{self._solution_count}")
            for d in range(self._num_days):
                if (self._solution_count <= self._solution_limit):
                    print(f"Day_{d}")
                    for n in range(self._num_nurses):
                        is_working = False
                        for s in range(self._num_shifts):
                            if self.value(self._shifts[(n, d, s)]):

```

```

        is_working = True
        if (self._solution_count<=self._solution_limit):
            print(f"_{n}_works_{shift}_{s}")
        if not is_working:
            if (self._solution_count<=self._solution_limit):
                print(f"_{n}_does_not_work")
        if (self._solution_count<=self._solution_limit):
            print()

    def solutionCount(self):
        return self._solution_count

# Display the first solution.
solution_limit = 1
solution_printer = NursesPartialSolutionPrinter(
    shifts, num_nurses, num_days, num_shifts, solution_limit
)

debut = time.perf_counter()
solver.solve(model, solution_printer)
fin = time.perf_counter()
duree = fin - debut

ch1 = "solutions_found"
l1 = len(ch1)
ch2 = "search_time"
l2 = len(ch2)

# Statistics.
print("\nStatistics")
print(f"_{solver.num_conflicts}_")
print(f"_{solver.num_branches}_")
print(f"_{(l1-l2-2)*\"_\", \"_:\",duree, \"s\"}")
print(f"_{solver.wall_time}_")
print(f"_{solution_printer.solutionCount()}")

if __name__ == "__main__":
    main()

```

9.2 Logigramme représentant le fonctionnement de l'interface tangible

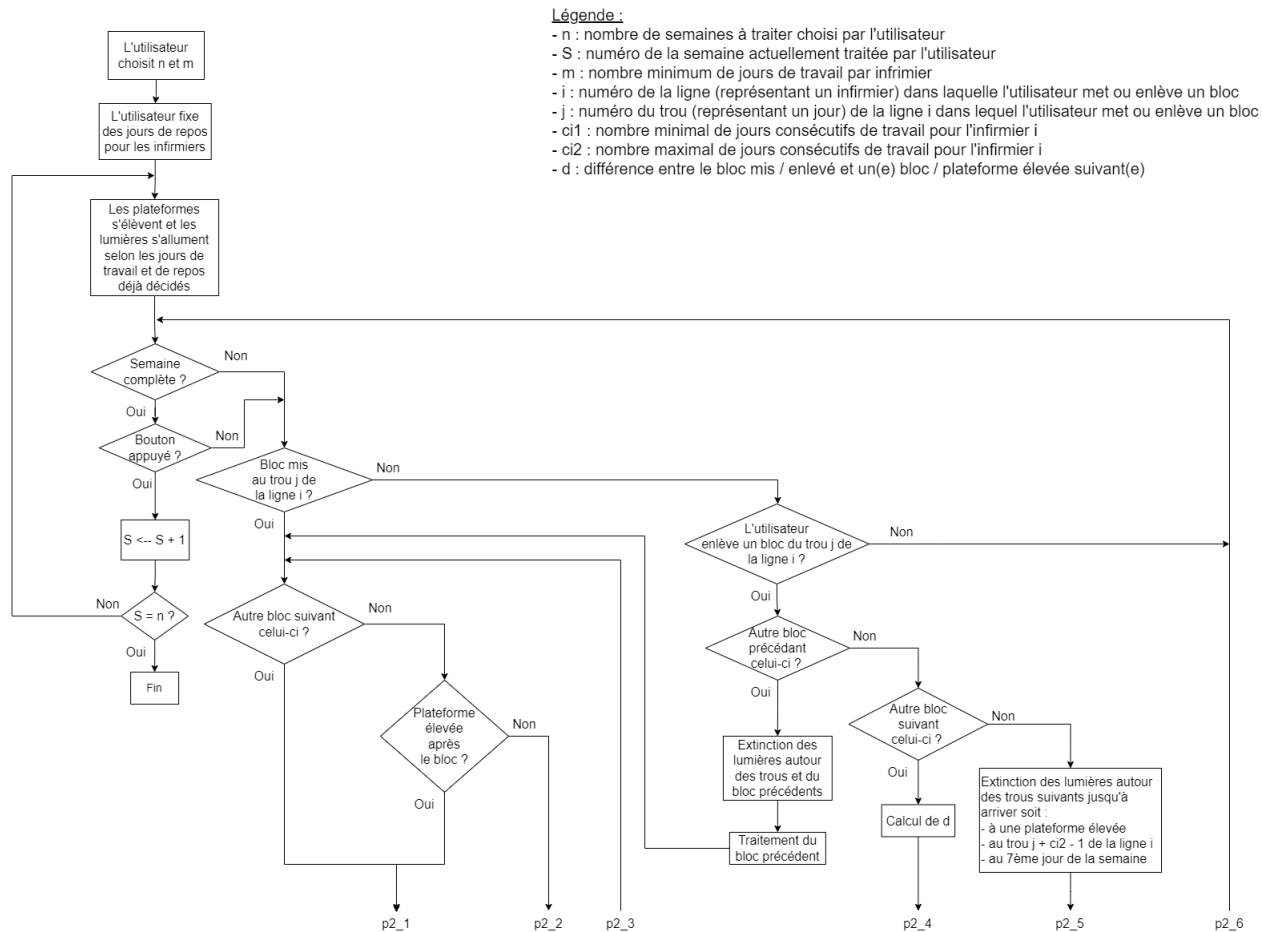


FIGURE 13 – Logigramme de l'interface tangible (page 1)

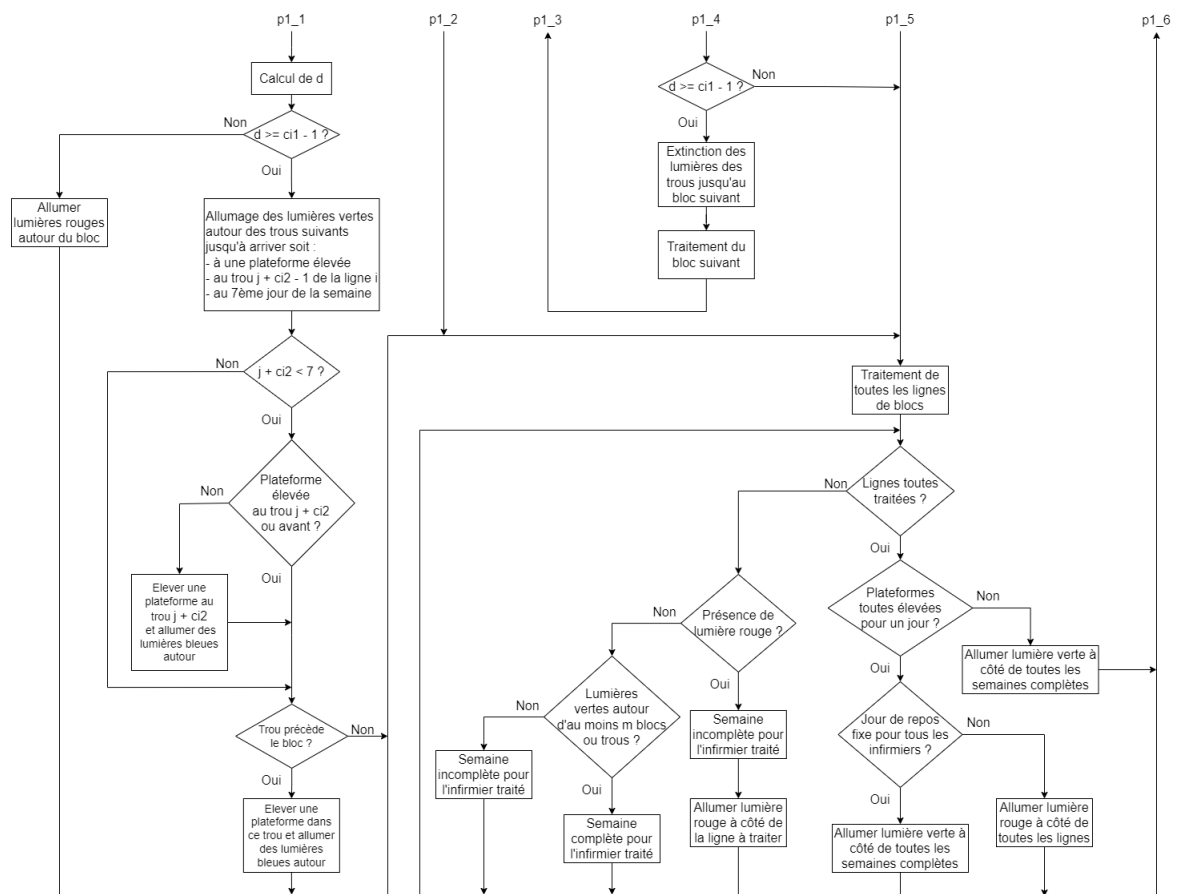


FIGURE 14 – Logigramme de l'interface tangible (page 2)